Duale Hochschule Baden-Württemberg Mosbach

Portfolio Web-Programmierung

CineRate

Studiengang Wirtschaftsinformatik

Profil: Industrie

Bearbeitungszeitraum: 01.09.2026 – 08.11.2026

Kurs	WWI25A
Anzahl der Mitglieder	3
Teamname	CineRate
Mitglieder	Lukas Krebelder - 3521843 Madlen Hahn - 7721417 Nico Weber - 1173793
Link zum Projektcode	https://github.com/lukaskrebelder/cinerate

Zusammenfassung

Gegenstand dieser Portfolioarbeit ist die Konzeption und prototypische Umsetzung von CineRate, einer Webanwendung zum Suchen, Entdecken, Verwalten und Bewerten von Filmen. Angemeldete Nutzer können Filme nach ihrem Namen suchen, nach Genres filtern und eine Detailseite mit weiteren Informationen aufrufen. Dort lassen sich Filme mit einem bis fünf Sternen bewerten und zu einer persönlichen Watchlist hinzufügen. Auf Basis der bewerteten Filme schlägt die Anwendung individuell passende Filme vor. Die Filmdaten stammen aus der externen Schnittstelle The Movie Database (TMDB), die über einen API-Key abgefragt wird, und werden bei Bedarf in einer eigenen PostgreSQL-Datenbank gespeichert. Die Anwendung ist als verteiltes System in mehreren Schichten aufgebaut: Das Backend stellt die REST-Schnittstelle bereit, und ein Backend-for-Frontend (BFF) bietet die Daten über REST und GraphQL an. Als Oberflächen dienen eine Single-Page-Application (SPA) mit vollem Funktionsumfang über REST (Representational State Transfer) und eine serverseitig gerenderte Multi-Page-Application (MPA), die GraphQL nutzt und nur einen abgesprochenen Teil der Funktionen abdeckt.

Abstract

The focus of this portfolio project is the design and prototyping of Ci-neRate, a web application for searching for, discovering, managing and rating films. Registered users can search for films by title, filter by genre and view a details page containing further information. There, films can be rated from one to five stars and added to a personal watchlist. Based on the rated films, the application suggests films tailored to the user's preferences. The film data is sourced from the external interface The Movie Database (TMDB), which is queried via an API key, and is stored in a dedicated PostgreSQL database where required. The application is structured as a distributed system comprising several layers: the backend provides the REST interface, and a Backend-for-Frontend (BFF) serves the data via REST and GraphQL. The user interfaces consist of a single-page application (SPA) with a full range of functions via REST (Representational State Transfer) and a server-side rendered multi-page application (MPA), which uses GraphQL and covers only a predefined subset of the functions.

Inhaltsverzeichnis

Abkürzungsverzeichnis II

Abbildungsverzeichnis III

1 Einführung in das Projekt 1

1.1 Motivation 1

1.2 Funktionen der Webanwendung 1

2 API-Design 6

3 Implementierung 9

3.1 Schichtenarchitektur im Backend 9

3.2 Zufälliges Token statt JWT 10

3.3 TMDB-Client mit Wiederholung und Cache 11

3.4 Filmvorschläge ohne Speichern: Round-Robin-Fehler und Fix 13

3.5 Tests: Was sie gefunden haben und was nicht 14

3.6 BFF und GraphQL: eine Schnittstelle für beide Oberflächen 15

3.7 SPA: Hash-Router, Zugriffsschutz und eine zentrale Stelle für Aufrufe 17

3.8 MPA: Seiten vom Server und Anmeldung per Cookie 18

3.9 Code-Review im Backend: fünf Schwächen, die kein Test zeigte 18

3.10 Fazit 20

4 Wissenschaftliche Reflexion 21

4.1 Wissenschaftliche Reflexion – Lukas Krebelder 21

4.2 Wissenschaftliche Reflexion – Madlen Hahn 23

4.3 Wissenschaftliche Reflexion – Nico Weber 26

Anhang ..... A

Abkürzungsverzeichnis

ADR	Architecture Decision Record
API	Application Programming Interface
BFF	Backend for Frontend
bzw.	beziehungsweise
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
EJS	Embedded JavaScript (Template-Engine)
ER	Entity-Relationship
FIFO	First In, First Out
GRASP	General Responsibility Assignment Software Patterns
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IMDB	Internet Movie Database
IP	Internet Protocol
ISO	International Organization for Standardization
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
KI	Künstliche Intelligenz
MPA	Multi-Page-Application
ms	Millisekunden
PB	Product-Backlog-Eintrag
PR	Pull Request
REST	Representational State Transfer
SHA	Secure Hash Algorithm
SPA	Single-Page-Application
SQL	Structured Query Language
TMDB	The Movie Database
URL	Uniform Resource Locator
US	User Story
XSS	Cross-Site Scripting
z. B.	zum Beispiel

Abbildungsverzeichnis

Abbildung 1 Registrierung und Anmeldung 2

Abbildung 2 Filmsuche mit Genre-Filter und Trefferliste 2

Abbildung 3 Detailseite mit Bewertung und Watchlist-Knopf 3

Abbildung 4 Kommentarbereich eines Films 3

Abbildung 5 Watchlist 4

Abbildung 6 Filmvorschläge auf der Seite „Für dich“ 4

Abbildung 7 Konto löschen 5

Abbildung 8 API-Design: Status, Anmeldung und Nutzerkonto 6

Abbildung 9 API-Design: Filme, Bewertungen und Watchlist 7

Abbildung 10 API-Design: Kommentare und Filmvorschläge 8

1 Einführung in das Projekt

1.1 Motivation

Für Filmbegeisterte gehören Plattformen wie IMDB zum Alltag, wenn es um Filme geht. Man kann dort nach Informationen zu einem Film suchen, Bewertungen schreiben oder lesen und Watchlists erstellen, um sich Filme zu merken. Da auch wir Enthusiasten des Films und Kinos sind haben wir uns für CineRate entschieden, eine Webanwendung, mit der sich ebenfalls Filme suchen, bewerten und auf einer persönlichen Watchlist sammeln lassen. Darüber hinaus bekommen Nutzer auf Basis der eigenen Bewertungen und Vorlieben passende Filme vorgeschlagen.

Ausschlaggebend hierfür war vor allem, dass alle im Team mit den fachlichen Anforderungen vertraut waren. Dadurch war es nahezu immer möglich, sich auf die technische Umsetzung zu konzentrieren, statt auf die Diskussion von Design und Funktionen. Hinzu kommt, dass die verschiedenen Funktionen einer solchen Webanwendung überschaubar genug sind, um sie in dem vorliegenden zeitlichen Rahmen klar in Muss-, Soll-, und Kann-Anforderungen aufzuteilen. Dies war wichtig, da wir nur einige wenige Wochen bis zur Abgabe des Projekts Zeit hatten und der Umfang realistisch bleiben sollte.

Zu guter Letzt lassen sich die vom Dozenten vorgegebenen Anforderungen anhand dieses Themas gut umsetzen. Konten, Bewertung und Watchlists erfordern eine eigene Datenerhaltung, während die Filmdaten wie Genre, Filmdetails und Empfehlungen aus der kostenlosen Schnittstelle The Movie Database (TMDB) stammen. Dadurch ergibt sich von selbst eine Architektur aus mehreren Komponenten statt eines Monolithen, doch dazu in Kapitel X mehr.

1.2 Funktionen der Webanwendung

CineRate besteht aus zwei Oberflächen, die auf dieselben Daten zugreifen: einer Single-Page-Application (SPA) mit vollem Funktionsumfang und einer Multi-Page-Application (MPA), die nur einen Teil davon abdeckt. Dieser Abschnitt stellt die Funktionen aus Sicht der Nutzer vor. Er zeigt sie anhand der SPA und geht nicht auf die technische Umsetzung ein.

Um CineRate zu nutzen, legen Nutzer zunächst ein Konto an. Dafür genügen Vor- und Nachname, eine E-Mail-Adresse und ein Passwort mit mindestens acht Zeichen (Abbildung 1, links). Nach der Anmeldung (Abbildung 1, rechts) stehen alle weiteren Seiten zur Verfügung, Gäste

werden stattdessen zur Anmeldung geleitet. Die Eingaben werden schon im Browser geprüft, und Fehler werden in verständlichen deutschen Meldungen erklärt.

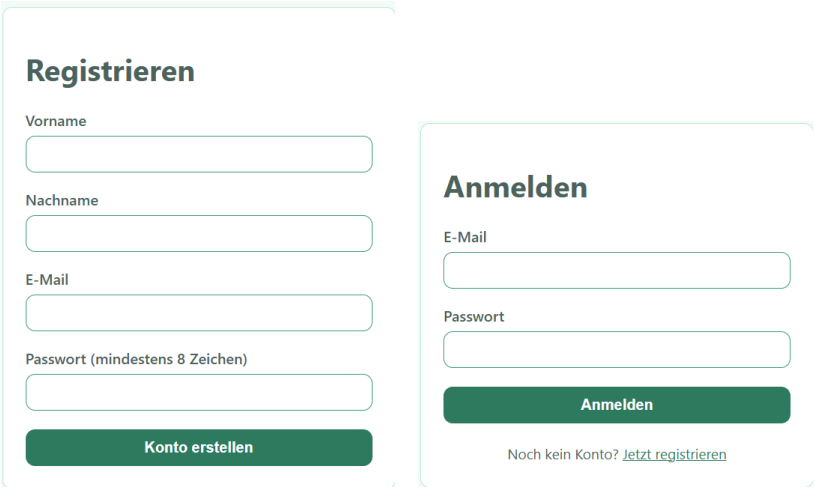


Abbildung 1 Registrierung und Anmeldung
(Quelle: Eigene Darstellung)

Angemeldete Nutzer finden Filme über die Suche. Sie können nach einem Titel suchen, nach einem Genre filtern und durch die Trefferliste blättern, ohne Eingabe erscheint eine Standardliste (Abbildung 2). Die Filmdaten stammen von der externen Schnittstelle The Movie Database (TMDB) und werden im Backend zwischengespeichert.

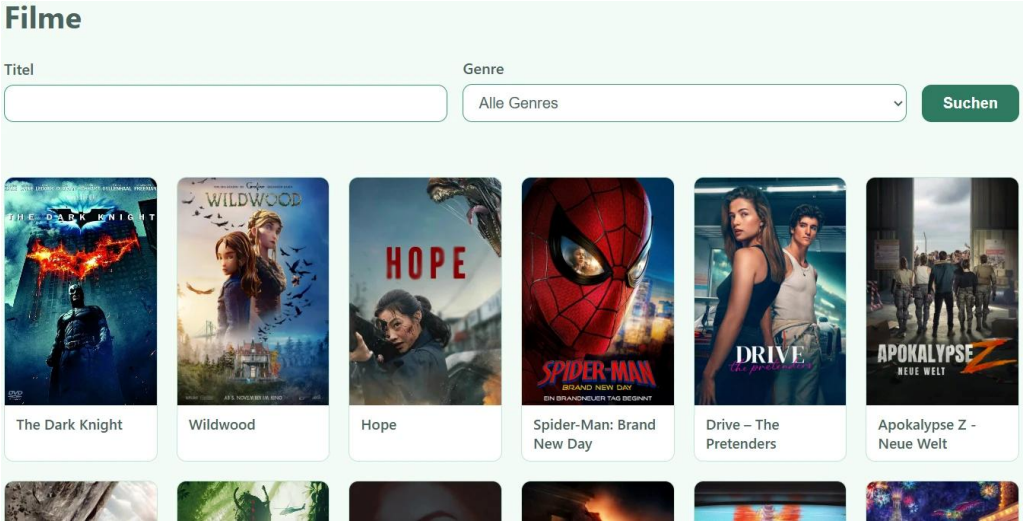


Abbildung 2 Filmsuche mit Genre-Filter und Trefferliste
(Quelle: Eigene Darstellung)

Ein Klick auf einen Film öffnet die Detailseite mit Poster, Genres, Erscheinungsjahr, Laufzeit, Altersfreigabe und Beschreibung (Abbildung 3). Dort bewerten Nutzer den Film mit einem bis

fünf Sternen und können ihre Bewertung später ändern. Angezeigt wird außerdem der Durchschnitt aller Bewertungen, deren Anzahl und die eigene Bewertung. Über einen Knopf wird der Film der persönlichen Watchlist hinzugefügt oder wieder von ihr entfernt.



Abbildung 3 Detailseite mit Bewertung und Watchlist-Knopf
(Quelle: Eigene Darstellung)

Unter den Filmdetails steht der Kommentarbereich (Abbildung 4). Kommentare dürfen bis zu 500 Zeichen lang sein und erscheinen mit dem neuesten zuerst, 20 pro Seite, zusammen mit Vornamen und Zeitpunkt. Eigene Kommentare lassen sich nachträglich bearbeiten, was durch den Hinweis „bearbeitet am“ kenntlich bleibt, oder nach einer Sicherheitsabfrage löschen. Fremde Kommentare können Nutzer nicht verändern. Das Backend kennt zusätzlich die Rolle Moderator, die auch fremde Kommentare löschen darf.



Abbildung 4 Kommentarbereich eines Films
(Quelle: Eigene Darstellung)

Auf der Seite „Meine Watchlist“ sehen Nutzer alle vorgemerkten Filme und können sie mit einem Klick wieder entfernen (Abbildung 5). Die Watchlist ist persönlich und für andere Nutzer nicht einsehbar.

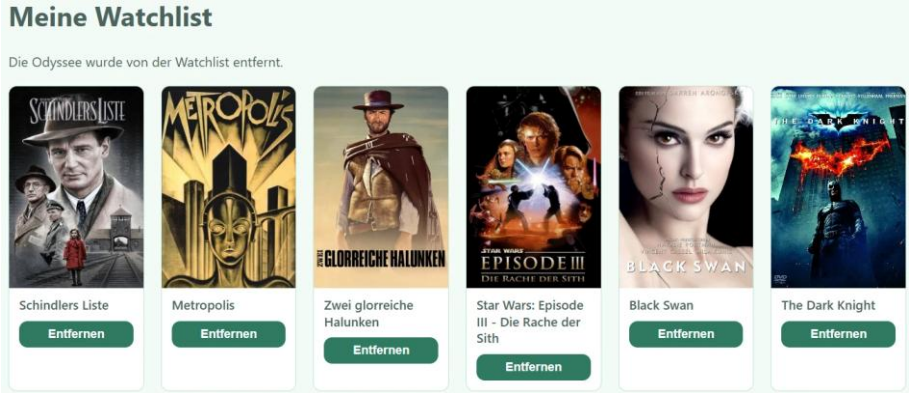


Abbildung 5 Watchlist
(Quelle: Eigene Darstellung)

Die Seite „Für dich“ schlägt bis zu zehn Filme vor (Abbildung 6). Grundlage ist das Lieblingsgenre eines Nutzers, also das Genre, in dem er die meisten Filme mit vier oder fünf Sternen bewertet hat. Bereits bewertete Filme werden nicht erneut vorgeschlagen. Haben Nutzer noch zu wenige Filme bewertet, weist die Seite darauf hin, dass zuerst mindestens drei Bewertungen nötig sind.

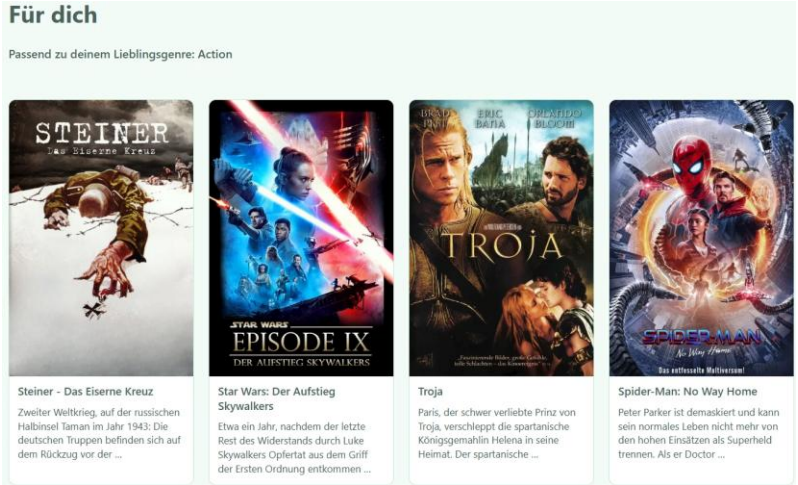


Abbildung 6 Filmvorschläge auf der Seite „Für dich“
(Quelle: Eigene Darstellung)

Unter „Mein Konto“ können Nutzer ihr Konto löschen (Abbildung 7). Damit werden auch alle zugehörigen Daten, also Bewertungen, Kommentare und Watchlist, unwiderruflich entfernt.

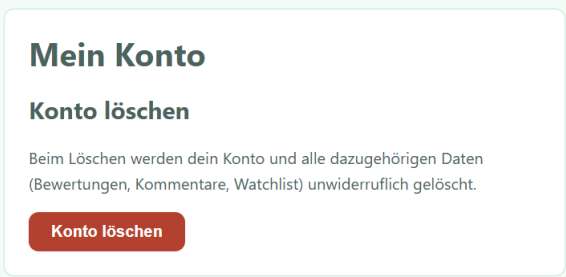


Abbildung 7 Konto löschen
(Quelle: Eigene Darstellung)

Die MPA deckt nur einen Teil dieser Funktionen ab. Sie bietet die Registrierung, die Anmeldung und Abmeldung sowie die Filmsuche mit Genre-Filter und Blätterfunktion, jeweils als serverseitig erzeugte Seiten. Detailseite, Bewertungen, Watchlist, Kommentare, Filmvorschläge und das Löschen des Kontos stehen dagegen nur in der SPA zur Verfügung. Das wurde so vereinbart. Die MPA muss lediglich die Grundfunktionen und einige wenige weitere Funktionen umsetzen muss, während die SPA den vollen Funktionsumfang erhält. Der Unterschied zeigt sich auch bei der Schnittstelle: Die MPA bezieht ihre Filmsuche über GraphQL, die SPA nutzt für alle Funktionen REST.

2 API-Design

Aufbau: Die Routen liegen im Backend unter `/api`. Das BFF bietet sie unter `/bff` per REST an und zusätzlich alle 17 per GraphQL unter `/bff/graphql` (7 Abfragen, 10 Mutationen). Die SPA nutzt nur REST. Die MPA holt Filmsuche und Filmdetails per GraphQL, Status, Registrieren, Anmelden und Abmelden per REST.

Intern: Das BFF schickt dem Backend zusätzlich den Header `X-Internal-Key`. Das betrifft nur BFF ↔ Backend, nicht SPA oder MPA.

Token: Routen mit „Token“ erwarten `Authorization: Bearer <token>` aus `POST /auth`. Fehlt das Token oder ist es ungültig, kommt 401 „Invalid token.“

Zustandslosigkeit: Jede Anfrage bringt alles mit, was der Server braucht (Token im Header, Parameter in URL oder Body). Der Server merkt sich zwischen zwei Anfragen keinen Ablauf; die Sitzungstabelle speichert nur, ob ein Token gültig ist, damit Abmelden sofort wirkt.

Fehler haben immer die Form `{"message": "…"}`; unten steht nur der Statuscode und der Text. Bei allen Routen gilt zusätzlich: 400 „Request body is not valid JSON.“ bei kaputtem JSON, 404 „Route not found.“ bei unbekannter Route, 500 „Internal server error.“ bei unerwarteten Fehlern und 503 „Database is not reachable.“ nur, wenn die Datenbank nicht erreichbar ist. Ist TMDB nicht erreichbar, kommt 503 „Movie data is currently unavailable.“

Method	Ressource	Data In	Data Out
Status zeigt, dass Backend und Datenbank laufen			
GET	`/status`	–	**200** `{"status": "ok", "nachricht": "CineRate-Backend läuft", "filmeInDb": 1240}`
Auth und Users US-2 · PB-01 bis PB-04			
POST	`/users`	`{"firstName": "Lena", "lastName": "Hartmann", "email": "lena.hartmann@example.com", "password": "Kino2026!"}`	**201** `{"message": "Account has been created.", "userId": "u_58213"}` **400** Invalid email or password (at least 8 characters). **409** An account with this email already exists.
POST	`/auth`	`{"email": "lena.hartmann@example.com", "password": "Kino2026!"}`	**200** `{"token": "a7f3k92xq1m8d5v0", "userId": "u_58213", "rolle": "Nutzer"}` rolle: „Nutzer“ oder „Moderator“ **400** Email and password are required. **401** Email or password is incorrect. **429** Too many failed login attempts. Please try again later. 429 nach 5 Fehlversuchen mit derselben E-Mail, 15 Minuten gesperrt
DELETE	`/auth` Token	–	**200** `{"message": "Logout successful."}`
DELETE	`/users/{userId}` Token	–	**200** `{"message": "Account has been deleted."}` **403** You can only delete your own account.

Abbildung 8 API-Design: Status, Anmeldung und Nutzerkonto
(Quelle: Eigene Darstellung, Stand 07.11.2026)

Method	Ressource	Data In	Data Out
Movies	US-5 · PB-06 bis PB-09, PB-17 · genre: deutscher TMDB-Name, unbekannt ergibt 400 · page 1 bis 500 · fsk wird noch nicht ausgewertet (PB-30) · Felder ohne Daten bei TMDB sind null, avgRating ist null ohne Bewertung · ungültige IDs ergeben 404		
GET	`/movies` Token	Query: `query, genre, page (alle optional)`	**200** `{"page": 1, "totalPages": 5, "movies": [{"movieId": 157336, "title": "Interstellar", "posterUrl": "https://image.tmdb.org/..."}]}` **400** Invalid filter value. **503** Movie data is currently unavailable.
GET	`/movies/{movieId}` Token	–	**200** `{"movieId": 157336, "title": "Interstellar", "description": "Astronauten suchen eine neue Heimat …", "genres": ["Drama", "Science Fiction"], "fsk": 12, "releaseYear": 2014, "runtime": 169, "posterUrl": "https://image.tmdb.org/...", "avgRating": 4.5}` **404** No movie found for this ID. **503** Movie data is currently unavailable.
Ratings	US-4 · PB-18 bis PB-20 · 201 = neue Bewertung, 200 = eigene Bewertung geändert · ownStars ist null, solange der Nutzer nicht bewertet hat		
POST	`/movies/{movieId}/ratings` Token	`{"stars": 5}`	**201 / 200** `{"message": "Your rating has been saved.", "stars": 5, "avgRating": 4.5}` bei 200: „Your rating has been updated.“ **400** Please choose 1 to 5 stars. **404** No movie found for this ID.
GET	`/movies/{movieId}/ratings` Token	–	**200** `{"avgRating": 4.5, "ratingCount": 2, "ownStars": 5}` **404** No movie found for this ID.
Watchlist	US-3 · PB-21, PB-22 · zuletzt hinzugefügte Filme zuerst		
GET	`/watchlist` Token	–	**200** `{"movies": [{"movieId": 157336, "title": "Interstellar", "posterUrl": "https://image.tmdb.org/..."}]}`
POST	`/watchlist` Token	`{"movieId": 157336}`	**201** `{"message": "Movie was added to your watchlist."}` **404** No movie found for this ID. **409** Movie is already on your watchlist.
DELETE	`/watchlist/{movieId}` Token	–	**200** `{"message": "Movie was removed from your watchlist."}` **404** Movie is not on your watchlist.

Abbildung 9 API-Design: Filme, Bewertungen und Watchlist
(Quelle: Eigene Darstellung, Stand 07.11.2026)

Method	Ressource	Data In	Data Out
Comments PB-32 bis PB-35 · Text 1 bis 500 Zeichen · neueste zuerst, 20 pro Seite · bearbeiten nur der Autor, löschen Autor oder Moderator · updatedAt ist null, solange nicht bearbeitet			
POST	`/movies/{movieId}/comments` Token	`{"text": "Spannend bis zur letzten Minute!"}`	**201** `{"message": "Your comment has been posted.", "commentId": "c_1042"}` **400** Comment must have 1 to 500 characters. **404** No movie found for this ID.
GET	`/movies/{movieId}/comments` Token	Query: `page (optional)`	**200** `{"page": 1, "totalPages": 1, "comments": [{"commentId": "c_1042", "userId": "u_58213", "firstName": "Lena", "text": "Spannend bis zur letzten Minute!", "createdAt": "2026-10-01T18:30:00Z", "updatedAt": null}]}` **400** Invalid page. **404** No movie found for this ID.
PUT	`/movies/{movieId}/comments/{commentId}` Token	`{"text": "Spannend bis zum Schluss!"}`	**200** `{"message": "Your comment has been updated."}` **400** Comment must have 1 to 500 characters. **403** You can only edit your own comments. **404** No comment found for this ID.
DELETE	`/movies/{movieId}/comments/{commentId}` Token	–	**200** `{"message": "Comment was deleted."}` **403** You can only delete your own comments. **404** No comment found for this ID.
Recommendations US-1 · PB-26 · mindestens 3 bewertete Filme, davon einer mit 4 oder 5 Sternen · bis zu 10 Filme, bereits bewertete ausgeschlossen			
GET	`/recommendations` Token	–	**200** `{"favoriteGenre": "Science Fiction", "movies": [{"movieId": 329865, "title": "Arrival", "description": "…", "posterUrl": "https://image.tmdb.org/..."}]}` **400** Please rate at least 3 movies first. **400** Please rate at least one movie with 4 or 5 stars. **404** No new recommendations available. **503** Movie data is currently unavailable.

Abbildung 10 API-Design: Kommentare und Filmvorschläge
(Quelle: Eigene Darstellung, Stand 07.11.2026)

3 Implementierung

Die Entscheidungen im Team haben wir als Architecture Decision Records (ADR) festgehalten, kurze Dokumente mit Kontext, Alternativen, Entscheidung und Folgen. Verweise wie ADR-05 meinen den ADR-Satz fürs Portfolio (ADR-01 bis ADR-07, Anlage `ADRs-Portfolio.pdf`), nicht den Satz für Schalles mit eigener Nummerierung. Die wichtigsten Gründe stehen in den Abschnitten selbst, sodass das Kapitel auch ohne die ADRs lesbar ist.

CineRate besteht aus vier Teilen, jeweils mit eigenem Repository und eigener Anwendung auf Heroku: zwei Oberflächen (SPA und MPA, Nico), das BFF (Madlen) und das Backend mit der PostgreSQL-Datenbank (Lukas). Beide Oberflächen sprechen nur mit dem BFF, das BFF nur mit dem Backend, und nur das Backend greift auf die Datenbank und auf TMDB zu (ADR-01).

3.1 Schichtenarchitektur im Backend

Am 14.10.2026 haben wir das Backend von Routen mit eigenem SQL auf drei Schichten umgebaut (Route, Service, Repository), damit sich die Fachregeln ohne Datenbank testen lassen (ADR-05).

Problem. Am 12.10.2026 standen sieben SQL-Abfragen direkt in Routen und Middleware. Fachregeln und SQL waren vermischt, jeder Test brauchte eine Datenbank. In Sprint 1 hatten wir eine Service-Schicht noch als zu viel Struktur abgelehnt (ADR-03). Mit Filmvorschlägen und TMDB-Cache kamen aber Abläufe dazu, die nicht mehr in eine Route passten.

Entscheidung. Die Abhängigkeiten zeigen nur in eine Richtung: Route → Service → Repository. Die Route kennt HTTP, der Service die Regeln, das Repository das SQL. Services sind Fabrikfunktionen und bekommen ihre Abhängigkeiten als Parameter, ein Test kann also ein Ersatz-Repository übergeben. Verworfen haben wir, alles in den Routen zu lassen (doppeltes SQL, nur mit Datenbank testbar), ein Framework wie NestJS (keine Vorerfahrung) und Services als Klassen (passt nicht zum Funktionsstil in `lib/`). Das Repository übersetzt auch Datenbankfehler, zum Beispiel `23505` (E-Mail gibt es schon) in einen `AppError` mit 409, damit der Service PostgreSQL nicht kennen muss.

Manche Regeln liegen direkt in der Datenbank: Beim Löschen eines Kontos löscht das Repository nur die Zeile in `Nutzer`. Sitzungen, Bewertungen, Kommentare und Watchlist entfernt `ON DELETE CASCADE` in derselben Anweisung mit.

Beleg. `test/userService.test.js` testet den Service mit Ersatz-Repository und ohne Datenbank, zum Beispiel dass ein zu kurzes Passwort 400 ergibt und nichts gespeichert wird. Stand 07.11.2026: 199 Tests in 27 Dateien, alle grün (`node --test`). Die Kaskade haben wir auf Heroku geprüft, nur die Watchlist nicht über die API.

Reflexion. Unsere Entscheidung aus Sprint 1 war zu kurz gedacht. Wir haben sie zurückgenommen, sobald Filmvorschläge und Cache eigene Logik brauchten, und dafür mehr Dateien und Zeit für den Umbau in Kauf genommen. Unerwartete Fehler haben wir lange als 503 gemeldet, obwohl 500 richtiger wäre; behoben haben wir das erst beim Code-Review am 06.11.2026 (Abschnitt 3.9). Im nächsten Projekt würden wir die Schichten von Anfang an anlegen, sobald mehr als eine User Story eigene Fachregeln hat.

3.2 Zufälliges Token statt JWT

Am 06.10.2026 haben wir entschieden, die Anmeldung mit einem zufälligen Token und einer Sitzungstabelle statt mit einem JWT umzusetzen, weil Abmelden und Account löschen sofort wirken müssen (ADR-04).

Problem. Laut den Acceptance Criteria zu US-2 macht Abmelden das Token ungültig und das Löschen eines Accounts alle seine Tokens. Geplant war ursprünglich ein JWT, wie im Beispielprojekt von Herrn Hartwig.

Entscheidung. Bei der Anmeldung prüft das Backend das Passwort gegen den bcrypt-Hash in `Nutzer` und erzeugt ein Token aus 32 Zufallsbytes. In `Sitzung` speichern wir nur dessen SHA-256-Hash mit `nutzer_id` und Ablaufzeit (24 Stunden). Bei jeder geschützten Route hasht das Backend das mitgeschickte Token und sucht den Hash; ist er unbekannt oder abgelaufen, kommt 401. Abmelden löscht die Zeile, beim Löschen eines Accounts greift `ON DELETE CASCADE`. Ein JWT haben wir verworfen, weil es bis zum Ablauf gültig bleibt und eine Sperrliste den Vorteil der fehlenden Datenbankabfrage wieder aufheben würde. Ein Token im Klartext wäre bei einem Datenbankleck sofort nutzbar. Für den Token-Hash reicht SHA-256: Das Token hat 256 Bit Zufall, und der Hash muss bei gleichem Token gleich sein, damit die Datenbank direkt danach suchen kann. bcrypt nimmt bei jedem Aufruf ein neues Salt und ginge dafür nicht.

```
// lib/tokens.js
function hashToken(token) {
  return crypto.createHash("sha256").update(token).digest("hex");
}

function createToken() {
  const token = crypto.randomBytes(32).toString("hex");
  return { token, tokenHash: hashToken(token) };
}
```

Beleg. `test/tokens.test.js` prüft den Hash gegen den bekannten SHA-256-Wert für „abc“ und dass jedes Token anders ist. Auf Heroku wurde dasselbe Token am 24.10.2026 nach dem Abmelden und nach dem Löschen des Kontos mit 401 abgelehnt (`Heroku-Test-24-10-2026.md`). Das hätte ein JWT ohne Sperrliste nicht geschafft.

Reflexion. Die sofortige Sperrbarkeit war uns wichtiger als eine gesparte Datenbankabfrage pro Anfrage. Abgelaufene Sitzungen wurden lange nur abgelehnt, aber nie gelöscht; seit dem Code-Review am 06.11.2026 löscht das Backend sie bei jeder erfolgreichen Anmeldung (Abschnitt 3.9). Außerdem haben wir die Abweichung vom JWT im Beispielprojekt nicht mit Herrn Hartwig abgesprochen.

3.3 TMDB-Client mit Wiederholung und Cache

Den Zugriff auf TMDB haben wir in einen Client mit Timeout und Wiederholung und einen Cache davor aufgeteilt. Eine Suche bei TMDB dauerte lokal 205 bis 432 ms, dieselbe Antwort aus dem Cache höchstens 0,1 ms.

Problem. TMDB ist unsere einzige externe Schnittstelle. Bewertet werden die Integration eines Fremdservices (mehr als ein GET, Fehlerbehandlung, Asynchronität) und eine Optimierung der Infrastruktur. Ein Ausfall von TMDB darf die Anwendung nicht blockieren, und gleiche Anfragen sollen TMDB nicht jedes Mal neu belasten.

Entscheidung. Der Client (`integrations/tmdbClient.js`) setzt pro Versuch ein Timeout von 5 Sekunden und versucht es höchstens dreimal, mit 200 ms und 400 ms Pause. Wiederholt wird nur bei Netzwerkfehler, Timeout, 429 und 5xx. Ein 404 geht sofort als 404 weiter, nach dem letzten Fehlversuch antwortet die API mit 503. Die URL wird nie geloggt, weil sie den API-Schlüssel enthält.

```
// integrations/tmdbClient.js (Ausschnitt aus request)
for (let attempt = 0; attempt <= retries; attempt++) {
  // Pause vor Versuch 2 und 3: 200 ms, 400 ms
  if (attempt > 0) await sleepFn(retryDelayMs * 2 ** (attempt - 1));
  try {
    const response = await fetchFn(url, {
      signal: AbortSignal.timeout(timeoutMs),
    });
    if (response.ok) return await response.json();
    lastError = new Error(`TMDB-Anfrage fehlgeschlagen: ${response.status}`);
    if (response.status === 404) { notFound = true; break; }
    // z. B. 401: nicht erneut versuchen
    if (!isRetryableStatus(response.status)) break;
  } catch (err) {
    lastError = err; // Netzwerkfehler oder Timeout: erneut versuchen
  }
}
```

Der Cache (`lib/ttlCache.js`, höchstens 500 Einträge, der älteste fliegt zuerst raus) liegt in `integrations/cachedTmdbClient.js` um den Client. Suche und Genre-Auswahl bleiben 10 Minuten gültig, Filmdetails und Empfehlungen 1 Stunde, die Genre-Liste 24 Stunden. Gleichzeitige gleiche Anfragen laden nur einmal, Fehler werden nicht gespeichert. Ein eigener Cache-Dienst hätte eine weitere Heroku-App gekostet, und Herr Hartwig hat am 14.10.2026 gesagt, dass das nicht nötig ist (ADR-06). In der Datenbank speichern wir einen Film erst, wenn jemand die Detailseite öffnet, ihn bewertet, kommentiert oder vormerkt. Diesen Ablauf haben wir am 22.10.2026 in die gemeinsame Funktion `ensureMovie` im `movieService` gezogen, statt ihn für die Bewertungen zu kopieren.

Beleg. `scripts/messung-cache.js` hat am 22.10.2026 in drei Läufen lokal gegen das echte TMDB gemessen: Die Suche dauerte beim ersten Aufruf 432, 206 und 205 ms, das Filmdetail 128, 25 und 29 ms, der zweite Aufruf jeweils 0,0 ms aus dem Cache. Auf Heroku dauerten die Filmvorschläge beim ersten Aufruf 153 ms, beim zweiten 69 ms (dort mit Datenbank und Netzwerk). Zwölf Tests in `test/tmdbClient.test.js` prüfen mit ausgetauschtem `fetch` die Wiederholung bei 500, 429 und Netzwerkfehler, keine Wiederholung bei 404 und die Pausen. Neun weitere Tests prüfen den Cache, fünf in test/movieService.test.js den Filmimport über `ensureMovie`.

Reflexion. Unsere erste Messung mit `curl` über die ganze Route (207 ms ohne, 64 ms mit Cache) enthielt auch Tokenprüfung und Datenbank, deshalb haben wir das Skript gebaut. Auch das zeigt nur die Größenordnung: lokal, eine Suche, ein Film. Der Cache liegt im Speicher einer Instanz und ist nach jedem Neustart leer. In der ersten Version wurde ein 404 von TMDB wie ein Ausfall wiederholt, das haben wir am 19.10.2026 korrigiert. Im schlechtesten Fall wartet eine Anfrage 15,6 Sekunden (3 × 5 s plus 0,6 s Pausen), das ist zu lang. Außerdem läuft der

Filmimport innerhalb der Anfrage. Zwei weitere Grenzen haben wir erst beim Code-Review am 06.11.2026 bewusst gesehen. Für Filmdetails bringt der Cache wenig, weil `ensureMovie` zuerst die Datenbank fragt und sie nach dem ersten Import die Rolle des Zwischenspeichers übernimmt; spürbar wirkt der Cache bei Suche, Genres und Vorschlägen, die nie gespeichert werden. Und weil auch das Lesen von Bewertungen und Kommentaren `ensureMovie` nutzt, kann ein GET einen Film speichern. Das widerspricht dem Grundsatz, dass GET nichts verändert; abgeschwächt wird es dadurch, dass der Import per Upsert idempotent ist. Sauberer wäre, beim Lesen nur die Datenbank zu fragen.

3.4 Filmvorschläge ohne Speichern: Round-Robin-Fehler und Fix

Problem. Für US-1 (PB-26) schlägt CineRate ab drei bewerteten Filmen weitere Filme vor. Das Lieblingsgenre ergibt sich aus den Filmen mit 4 oder 5 Sternen (bei Gleichstand entscheidet der zuletzt bewertete Film, danach das Alphabet), die Vorschläge kommen aus den TMDB-Empfehlungen zu diesen Quellfilmen.

Entscheidung. Wir speichern keine Vorschläge (Annahme A1), sondern geben die TMDB-Daten direkt weiter; die Last fängt der Cache aus Abschnitt 3.3 ab. Der Service fragt die Empfehlungen von bis zu zehn Quellfilmen gleichzeitig mit `Promise.all` ab und filtert bewertete Filme und Dubletten heraus. Ein 404 einer Quelle wird übersprungen, ein 503 bricht die Anfrage ab, damit keine unvollständige Liste als vollständig erscheint (A4).

Die erste Version (Commit df4b80c, 24.10.2026) ging die Quellfilme nacheinander durch, sodass oft der zuletzt bewertete Film alle zehn Plätze belegte. Am 28.10.2026 haben wir auf Round Robin umgestellt (Commit d91409e): erst der erste Vorschlag jeder Quelle, dann der zweite und so weiter. Zufälliges Mischen haben wir verworfen, weil es nicht reproduzierbar und damit nicht testbar wäre.

```
const seen = new Set();
const movies = [];
for (
  let platz = 0;
  movies.length < MAX_RECOMMENDATIONS &&
    lists.some((results) => platz < results.length);
  platz++
) {
  for (const results of lists) {
    const candidate = results[platz];
    if (!candidate || alreadyRated.has(candidate.id)
        || seen.has(candidate.id)) continue;
    seen.add(candidate.id);
    movies.push(toRecommendation(candidate));
    if (movies.length === MAX_RECOMMENDATIONS) break;
  }
}
```

Beleg. Mit dem Fix kamen zwei Unit-Tests dazu (135 → 137): Die Vorschläge kommen reihum aus mehreren Quellen, und es gibt höchstens zehn TMDB-Aufrufe. Auf Heroku haben wir am 28.10. erst mit einem Quellfilm (The Matrix), dann zusätzlich mit Inception und Interstellar getestet: 6 der 10 Vorschläge waren neu und kamen von den neuen Quellen.

Reflexion. Der Fehler ist durch unsere Tests gerutscht, weil wir die erwartete Reihenfolge so aufgeschrieben hatten, wie die erste Version sie lieferte. Auch den Heroku-Test vom 24.10. haben wir so abgehakt. Bemerkt hat ihn erst Nico beim Ausprobieren. Beim nächsten Mal schreiben wir vorher auf, was der Nutzer erwartet („etwas aus allen meinen Lieblingsfilmen“), und leiten den Test daraus ab.

3.5 Tests: Was sie gefunden haben und was nicht

Problem. Wir wollten bei jeder Änderung schnell sehen, ob etwas kaputt ist, ohne Datenbank und Internet, und trotzdem wissen, ob das System auf Heroku läuft.

Entscheidung. Wir testen auf zwei Ebenen. 199 Unit-Tests in 27 Dateien (Stand 07.11.2026, `node --test`), in denen Datenbank und TMDB durch Attrappen ersetzt sind, laufen bei jedem Push und Pull Request auf `main` in GitHub Actions; Heroku deployt erst bei grüner CI. Dazu kommen PowerShell-Skripte, die wir nach dem Deploy von Hand gegen das echte Backend laufen lassen (24.10.: 38 von 38 und 18 von 18 für die Vorschläge, 28.10.: 9 von 9, 31.10.: 37 von 37 Prüfungen bestanden). Tests gegen eine echte Datenbank fehlen aus Zeitgründen.

Fall 1: Zeitformat der Kommentare. In `db/commentRepository.js` stand im Formatmuster von `to_char` ein Anführungszeichen zu viel. Dadurch gab PostgreSQL für `createdAt` und `updatedAt` den Text `YYYY-MM-DDTHH24:MI:SSZ` statt eines Zeitpunkts aus. Die Unit-Tests waren

grün, weil der Ersatz-Pool kein SQL ausführt; aufgefallen ist es Madlen im BFF. Der Fix vom 31.10. (Commit efb2b6f) ist eine Zeile, bestätigt hat ihn erst der Heroku-Lauf am selben Tag:

```
-const ISO = `"YYYY-MM-DD"T"HH24:MI:SS"Z"`;
+const ISO = `YYYY-MM-DD"T"HH24:MI:SS"Z"`;
```

Fall 2: Rote CI nach `git stash pop`. Am 02.11. hatte ein `git stash pop` einen alten Stand zurückgeholt, den wir mit `git add .` samt Konfliktmarkern in der Workflow-Datei mitcommittet haben. Die CI auf `main` wurde rot, Heroku hat deshalb nicht deployt, und PR #23 ging einen Tag später live. Behoben haben wir das am 03.11. mit PR #24.

Reflexion. Unit-Tests mit Attrappen prüfen unsere Logik, aber nicht das, was die Attrappen ersetzen: SQL, Kaskade und TMDB sind nur durch die Heroku-Läufe von Hand belegt. Beim nächsten Mal würden wir die Repositories in der CI gegen eine echte Datenbank testen und vor jedem Commit `git status` und `git diff` ansehen.

3.6 BFF und GraphQL: eine Schnittstelle für beide Oberflächen

Problem. SPA und MPA sollen dieselben Daten bekommen, ohne dass sie das Backend, die Datenbank oder TMDB kennen. Dazu verlangt die Aufgabenstellung, die Daten über REST und über GraphQL anzubieten. Wir mussten entscheiden, wo die Fachregeln liegen und wie wir zwei Zugriffsarten anbieten, ohne die Regeln doppelt zu bauen.

Entscheidung. Das BFF (Backend for Frontend, umgesetzt von Madlen) hat keine Datenbank, ruft TMDB nicht auf und enthält keine Fachregeln (ADR-01, ADR-02). Unter `/bff/...` reicht es jede Anfrage an die passende Backend-Route weiter, mit den dafür vorgesehenen Parametern und Feldern und dem `Authorization`-Header (bei den Kommentaren zum Beispiel nur `text` und `page`). Status und Antwort des Backends gibt es zurück; ist das Backend nicht erreichbar, antwortet das BFF mit 503. Daneben gibt es unter `/bff/graphql` einen GraphQL-Endpunkt (Pakete `graphql` und `graphql-http`; Node.js hat Herr Hartwig am 14.10.2026 für GraphQL zugelassen). Das Schema hat sieben Abfragen (`filme`, `film`, `bewertungen`, `watchlist`, `vorschlaege`, `kommentare`, `status`) und zehn Mutationen (`filmBewerten`, `filmVormerken`, `vonWatchlistEntfernen`, `filmKommentieren`, `kommentarBearbeiten`, `kommentarLoeschen`, `registrieren`, `anmelden`, `abmelden`, `kontoLoeschen`). Die Resolver rufen das Backend ebenfalls per REST auf, die Fachregeln bleiben also an einer Stelle. Weil GraphQL Fehler sonst mit HTTP 200 meldet, setzen wir bei jedem Fehler `extensions.status` auf den Statuscode des Backends (503: nicht erreichbar, 502: ungültige Antwort).

Verworfen haben wir zwei Alternativen: Das BFF ruft TMDB selbst auf und schickt die Daten zum Speichern an das Backend (ADR-01, Teilentscheidung „Wer ruft die TMDB-API auf?“, Variante A; Schlüssel, Cache und Speichern lagen dann an zwei Orten), und GraphQL direkt auf der Datenbank (das BFF soll keinen Datenbankzugriff haben, die Regeln gäbe es doppelt).

```
// graphql/resolvers.js: gemeinsamer Aufruf für die neueren Resolver
async function requestBackend(
  path,
  { method = "GET", body, authorization, fallbackMessage },
) {
  // ... Header setzen, fetch auf das Backend
  // Backend nicht erreichbar → 503 "Backend is not reachable."
  // Antwort kein JSON      → 502 "Invalid response from backend."
  if (!backendResponse.ok) {
    throw backendError(
      backendResponse.status,
      responseData.message || fallbackMessage,
    );
  }
  return responseData;
}
```

Beleg. Das BFF hat 78 Unit-Tests in neun Dateien (REST-Routen je Bereich und GraphQL), darunter Fälle mit nicht erreichbarem Backend, 404 des Backends und ungültigen Abfragen. Sie laufen in einer eigenen CI. Den Weg über das BFF hat Madlen nach jedem Feature von Hand geprüft (lokales BFF gegen das Backend auf Heroku): am 23.10. Detailseite und Bewertungen, am 24.10. GraphQL mit den Statuscodes, am 25.10. die Watchlist, am 27.10. die Vorschläge und am 31.10. die Kommentare. Diese Läufe sind nicht in den Skripten dokumentiert.

Reflexion. Das BFF hält das Backend frei von Oberflächenfragen, bringt selbst aber wenig: Fast jede Route ist ein Weiterleiter. Das hat zwei Kosten. Erstens muss eine Änderung der API an mehreren Stellen nachgezogen werden (Backend-Route, REST-Route, Resolver, Schema). Zweitens wiederholt jede REST-Route die Fehlerbehandlung („`Backend is not reachable`“ steht in sieben der acht Routendateien); den eigentlichen Aufruf übernimmt seit dem 06.11. die gemeinsame Funktion in `lib/backend.js`. In den GraphQL-Resolvern gibt es dafür `requestBackend`, `filme` und `film` nutzen aber noch ihre eigene Kopie, weil sie vor der Hilfsfunktion entstanden sind. Beim nächsten Mal bauen wir die gemeinsame Funktion zuerst.

Ein zweiter Fehler betrifft die Abstimmung. Das Feld `extensions.status` kam am 24.10. hinzu, die MPA nutzt GraphQL aber seit dem 20.10. Sie liest den Status deshalb aus dem Fehlertext (zum Beispiel „Invalid token.“) und würde brechen, wenn das Backend einen Text ändert. Außerdem nutzen wir GraphQL kaum so, wie es gedacht ist: Jede Abfrage ruft genau einen Backend-Endpunkt ab, Daten aus mehreren Quellen werden nicht zusammengeführt.

3.7 SPA: Hash-Router, Zugriffsschutz und eine zentrale Stelle für Aufrufe

Problem. Die SPA (Nico) lädt die Seite einmal und tauscht danach nur den Inhalt. Dabei mussten wir drei Dinge lösen: Welche Ansicht zeigen wir zu welcher Adresse, wer darf welche Ansicht sehen, und was passiert, wenn die Sitzung abläuft, während jemand die Seite offen hat.

Entscheidung. Die SPA besteht aus Vanilla-JavaScript, HTML und CSS und spricht nur mit dem BFF. Ein Router ordnet Adressen hinter dem `#` Ansichten zu; jede Route trägt, ob sie nur für angemeldete Nutzer (`geschuetzt`) oder nur für Gäste (`nurGaeste`) gilt. Die Prüfung ist eine reine Funktion (`pruefeZugriff`), die ohne Browser getestet werden kann. Alle Aufrufe laufen über eine Funktion `anfrage()` in `api.js`. Sie hängt das Token an, meldet Netzfehler als eigenen Fehler und ruft bei einem 401 auf einer geschützten Route einen Handler auf, der abmeldet und zur Anmeldung wechselt. So muss keine Ansicht selbst auf eine abgelaufene Sitzung reagieren. Das Token liegt im `localStorage`; die Funktionen dafür nehmen einen Speicher als Parameter, damit Tests ein Ersatzobjekt übergeben können. Nach der Anmeldung speichert die SPA außer dem Token auch die Rolle aus der Login-Antwort (Nutzer oder Moderator). Moderatoren sehen bei fremden Kommentaren einen Löschen-Knopf; ob das Löschen erlaubt ist, prüft das Backend bei jedem Aufruf.

```
if (!antwort.ok) {
  if (geschuetzt && antwort.status === 401) sitzungsHandler();
  throw new ApiFehler(antwort.status, inhalt.message ?? "");
}
```

Der Zusatz `geschuetzt` ist nötig, weil ein 401 beim Login nur „Zugangsdaten falsch“ heißt und kein Sitzungsende ist.

Beleg. Die SPA hat 269 Unit-Tests in 25 Dateien, die in einer eigenen CI laufen. Sie prüfen die Hilfsfunktionen (Router, Zugriffsschutz, Suchpfad, Token, Formularprüfung, Hilfsfunktionen der Ansichten) und die Aufrufe an das BFF, jeweils ohne Browser.

Reflexion. Dass die Tests ohne Browser laufen, hat einen Preis: Wie die Seiten tatsächlich aussehen und sich verhalten, prüfen sie nicht. Das Token im `localStorage` kann von JavaScript gelesen werden, ein Cross-Site-Scripting-Fehler könnte es also stehlen. Das Risiko verringern wir, indem Namen und Kommentartexte anderer Nutzer nur mit textContent und nie als HTML eingesetzt werden. Die MPA schneidet hier besser ab (Abschnitt 3.8). Die BFF-Adresse steht fest in `api.js`, nicht in einer Umgebungsvariablen, und die SPA nutzt nur REST, GraphQL kommt bei uns nur in der MPA vor.

3.8 MPA: Seiten vom Server und Anmeldung per Cookie

Problem. Die Aufgabenstellung verlangt neben der SPA eine MPA, bei der jede Seite als eigenes HTML-Dokument vom Server kommt. Die MPA (Nico) sollte nur mit dem BFF sprechen und das Token vor dem JavaScript des Browsers verbergen.

Entscheidung (ADR-07). Die MPA ist ein kleiner Express-Server, der Seiten aus EJS-Vorlagen erzeugt. Nach der Anmeldung legt der Server das Token in ein Cookie (`httpOnly`, `secure` in Produktion, `sameSite: lax`, 24 Stunden wie das Token), liest es bei jedem Seitenaufruf und schickt es als `Authorization: Bearer` an das BFF. Lehnt das BFF das Token ab (401), löscht die MPA das Cookie und leitet zur Anmeldung weiter, mit einem Hinweis. Anmeldung und Abmeldung antworten nach Erfolg mit einer Weiterleitung (Status 303), damit ein Neuladen das Formular nicht erneut absendet. Aufrufe an das BFF haben ein Timeout von 5 Sekunden. Verworfen haben wir statische Seiten mit `fetch` im Browser (das wäre eine zweite SPA) und den Zugriff direkt auf das Backend (ADR-01). Außerdem soll JavaScript im Browser das Token nicht lesen können, deshalb liegt es in einem HttpOnly-Cookie.

```
function cookieOptionen() {
  return { httpOnly: true, secure: process.env.NODE_ENV === "production",
           sameSite: "lax", path: "/" };
}
```

Beleg. Die MPA hat 136 Unit-Tests in elf Dateien (Routen für Registrierung, Anmeldung, Suche und Detailseite, Formularprüfung, Fehlertexte, Menü, BFF-Aufrufe) und eine eigene CI.

Reflexion. Die MPA muss nicht alle Funktionen der SPA bieten, sie soll das MPA-Prinzip zeigen (ADR-07). Umgesetzt sind wie geplant Registrieren, Anmelden, Abmelden, die Filmsuche und die Detailseite (Stand 06.11.2026). Dazu kommen drei Grenzen: Der Schutz vor Cross-Site-Request-Forgery beruht nur auf `sameSite: lax`, die Statuscodes liest die MPA aus Fehlertexten (Abschnitt 3.6), und bei der Registrierung fehlt die Weiterleitung noch. Ein Neuladen nach dem Erfolg sendet das Formular dort erneut ab und führt zu einer Fehlermeldung, weil die E-Mail bereits vergeben ist (409).

3.9 Code-Review im Backend: fünf Schwächen, die kein Test zeigte

Am 06.11.2026 haben wir den Backend-Code vor der Abgabe noch einmal mit KI-Unterstützung durchgesehen (KI-Log A42 bis A46) und fünf Schwächen behoben, die alle Unit-Tests bestanden hatten.

Problem. Die Schwächen lagen nicht in der Fachlogik, sondern zwischen den Komponenten und in der Konfiguration. Genau dort ersetzen unsere Unit-Tests alles durch Attrappen (Abschnitt 3.5).

Entscheidung. Jede Änderung bekam einen eigenen Branch und einen Pull Request mit Review. Die Antworten an die Clients blieben gleich, nur neue Fehlercodes kamen dazu; BFF, SPA und MPA mussten deshalb nichts umbauen, außer dass das BFF den Schlüssel mitschickt. Die Schlüsselprüfung ist erst aktiv, wenn die Config Var `INTERNAL_API_KEY` auf dem Backend gesetzt ist. So konnten wir das Backend vor dem BFF ausliefern. Die Login-Sperre zählt pro E-Mail statt pro IP, weil alle Anfragen über das BFF kommen und das Backend für alle Nutzer dieselbe IP sieht.

Schwäche	Folge	Änderung
Ungültiges JSON und unbekannte Routen beantwortete Express mit einer HTML-Seite	Das BFF konnte die Antwort nicht lesen und meldete fälschlich 503 „Backend is not reachable.“	Zentrale Fehler-Middleware, Fehler immer als JSON; die 20 gleichen `try/catch`-Blöcke in den Routen entfallen (PR #28)
Jeder unerwartete Fehler wurde 503 „Database is not reachable.“	Ein Programmierfehler sah aus wie ein Datenbankausfall	500 als Standard, 503 nur bei Verbindungsfehlern zur Datenbank (PR #29, ADR-05)
Kein Listener für Fehler im Verbindungspool	Ein Verbindungsabbruch im Leerlauf hätte den ganzen Backend-Prozess beendet	`pool.on("error")` in `db/connection.js`
CORS sollte nur das BFF zulassen	CORS gilt nur für Browser, das Backend war für jeden direkt erreichbar	Gemeinsamer Schlüssel im Header `X-Internal-Key`, sonst 403 (ADR-01)
Beliebig viele Anmeldeversuche, abgelaufene Sitzungen blieben stehen	Passwörter ließen sich durchprobieren, `Sitzung` wuchs unbegrenzt	Sperre nach 5 Fehlversuchen pro E-Mail für 15 Minuten (429), Aufräumen bei jeder Anmeldung (ADR-04)

Beleg. 39 neue Unit-Tests (160 → 199). Neu sind HTTP-Tests gegen die gestartete App ohne Datenbank (`test/app.test.js`). Von Hand mit `curl` geprüft: ungültiges JSON ergibt 400, eine unbekannte Route 404, ein Aufruf ohne Schlüssel 403, jeweils als JSON. Seit dem 07.11.2026 läuft die Schlüsselprüfung auf Heroku. Ein Zwischenfall mit einem falschen Schlüssel führte dort zu 403 und zeigt, dass die Prüfung im Betrieb wirklich greift.

Reflexion. Die CORS-Annahme stand seit Sprint 1 in ADR-01 und im README, ohne dass wir sie je ausprobiert haben. Bei Sicherheitsannahmen schreiben wir künftig einen Test, der

den Angriff nachstellt, etwa einen Aufruf ohne Schlüssel. Die Review-Änderungen hat Lukas allein vorbereitet; das Team hat sie in den Pull Requests geprüft, die angepassten ADRs muss es noch bestätigen.

3.10 Fazit

Drei Dinge nehmen wir aus der Implementierung mit. Sie gelten für das ganze System aus SPA, MPA, BFF und Backend, am deutlichsten zeigen sie sich aber im Backend.

Erstens: Entscheidungen halten besser, wenn wir die Alternativen kannten. Die Sitzung mit gehashtem Token, der Cache um den TMDB-Client und die Kaskade beim Löschen haben wir gegen andere Lösungen abgewogen und nicht zurückgenommen. Bei den Schichten (Abschnitt 3.1) und beim gemeinsamen Filmimport (Abschnitt 3.3) haben wir dagegen zuerst zu knapp entschieden und später nachgearbeitet. Weitere Fehler sind die Filmauswahl bei den Vorschlägen (Abschnitt 3.4), wo wir die falsche Fachregel gebaut haben, weil wir sie nicht vorher als erwartetes Verhalten des Nutzers formuliert hatten, und die Abstimmung zwischen BFF und MPA beim Fehlerstatus (Abschnitt 3.6), die zwar abgesprochen, aber in der MPA nicht nachgezogen wurde.

Zweitens: Ein grüner Test belegt nur, was er wirklich ausführt. Unsere 199 Unit-Tests (Stand 07.11.2026, 27 Dateien) ersetzen Datenbank und TMDB durch Attrappen. Das hat uns schnelle Prüfungen und eine CI gebracht, aber das fehlerhafte Zeitformat der Kommentare und die falsche Auswahl der Vorschläge haben sie nicht gefunden. Beides fiel erst beim Ausprobieren auf, durch Madlen im BFF und durch Nico beim Testen, und die Heroku-Läufe haben es bestätigt. Die Tests der Zukunft würden das SQL gegen eine echte Datenbank prüfen und aus der erwarteten Wirkung für den Nutzer entstehen, nicht aus dem vorhandenen Code.

Drittens: Messen statt vermuten, und die Grenzen benennen. Den Cache haben wir gemessen (Abschnitt 3.3), aber nur lokal und auf Heroku mit drei Quellfilmen. Offengeblieben beziehungsweise spät erledigt sind: Die Tabelle `Sitzung` wird erst seit dem 06.11.2026 von abgelaufenen Einträgen bereinigt, der Cache verdrängt bei Vollwerden den ältesten Eintrag (FIFO) und wurde unter hoher Last nicht getestet, einmal gespeicherte Filmdaten werden nicht mit TMDB abgeglichen, im BFF wiederholt jede REST-Route die Fehlerbehandlung, und der Weg über BFF, SPA und MPA ist in unseren Backend-Tests nicht abgedeckt. Diese Punkte hätten wir mit mehr Zeit als Nächstes angepackt.

4 Wissenschaftliche Reflexion

4.1 Wissenschaftliche Reflexion – Lukas Krebelder

Reflexion der eigenen Arbeit

Im Team war ich Scrum Master und für das Backend und die Datenbank zuständig. Ich bin mit Java aus dem Studium in das Projekt gegangen. JavaScript hatten wir in der Vorlesung nur angeschnitten, Node.js, Express, PostgreSQL und Heroku waren für mich neu. Programmieren ist nicht meine Stärke, deshalb war das Backend für mich die größte Herausforderung im Projekt.

Als Scrum Master habe ich die Entscheidungen für unsere Teamtreffen vorbereitet und schriftlich festgehalten, damit alle auf demselben Stand arbeiten. Offene Fragen an die Dozenten habe ich gesammelt und gestellt. Für Code, Code-Reviews und Dokumentation habe ich Claude genutzt. Jeden Vorschlag habe ich selbst getestet, mit Unit-Tests, mit `curl` und auf Heroku, und erst nach dem Review gemergt. Die KI-Anteile sind im Code und im KI-Log gekennzeichnet.

Drei Entscheidungen im Backend möchte ich begründen. Erstens haben wir das Backend in drei Schichten aufgeteilt: Route, Service und Repository. In Sprint 1 hatten wir eine Service-Schicht noch als zu viel Struktur abgelehnt. Mit den Filmvorschlägen und dem Cache kamen aber Abläufe dazu, die nicht mehr in eine Route passten, und ohne Schichten ließen sich die Regeln nur mit Datenbank testen. Zweitens melden sich Nutzer mit einem zufälligen Token an, dessen Hash in einer Sitzungstabelle liegt, statt mit einem JWT. Nur so wirken Abmelden und Konto löschen sofort. Dafür kostet jede geschützte Anfrage eine Datenbankabfrage. Drittens liegt der Cache für TMDB im Backend und nicht in einem eigenen Dienst. Das spart eine weitere Heroku-App, dafür ist der Cache nach jedem Neustart leer.

Struktur und Organisation

Die Abstimmung im Team war aus meiner Sicht eine unserer größten Stärken. Wir haben uns täglich über WhatsApp ausgetauscht und uns mindestens einmal pro Woche in der Bibliothek getroffen. So konnten wir die Schnittstellen zwischen Backend, BFF und Oberflächen rechtzeitig festlegen.

Im Backend lief jede Änderung über einen eigenen Branch und einen Pull Request mit Review, bis zum 07.11.2026 waren das 32 Pull Requests. Die Tests laufen bei jedem Push in GitHub Actions, und Heroku deployt erst bei grüner CI.

Stärken und Schwächen

Als Stärke sehe ich, dass das Backend am Ende funktioniert hat, obwohl ich kaum Programmiererfahrung hatte. Es hat 199 Unit-Tests, alle grün, und läuft auf Heroku. Dazu kommt das Team. Absprachen haben immer funktioniert, und wir haben uns gegenseitig Fehler gezeigt. Madlen fand im BFF das falsche Zeitformat der Kommentare, Nico beim Ausprobieren die falsche Auswahl der Filmvorschläge.

Schwächen sehe ich vor allem am Anfang des Projekts. Die Portfolio-Guidelines kannten wir erst ab dem 12.10.2026, bis dahin waren MPA, GraphQL und eine Infrastruktur-Optimierung nicht geplant. Ohne KI hätte ich das Backend zeitlich nicht geschafft. Das hat einen Preis, denn Code, den ich nicht selbst geschrieben habe, verstehe ich nicht automatisch. Ich habe mir den Backend-Code deshalb vor der Präsentation Schritt für Schritt selbst erarbeitet.

Drei Rückschläge haben mir am meisten beigebracht. Erstens standen im Backend am 12.10.2026 sieben SQL-Abfragen direkt in Routen und Middleware, und ich musste am 14.10. auf die drei Schichten umbauen. Struktur kostet am Anfang wenig und später viel. Zweitens belegte in der ersten Version der Filmvorschläge vom 24.10. oft der zuletzt bewertete Film alle zehn Plätze. Meine Tests waren grün, weil ich die erwartete Reihenfolge so aufgeschrieben hatte, wie der Code sie lieferte. Gefunden hat den Fehler Nico, behoben habe ich ihn am 28.10. Drittens lagen mehrere Fehler außerhalb der Fachlogik: Konfliktmarker in der CI-Datei nach einem `git stash pop`, fünf Schwächen beim Code-Review, die alle Tests bestanden hatten, und ein falscher Schlüssel im Backend, durch den jede Anfrage des BFF mit 403 abgelehnt wurde. Alle diese Fälle lagen zwischen den Komponenten oder in der Konfiguration, also dort, wo meine Unit-Tests nichts prüfen.

Was ich anders machen würde

In einem ähnlichen Projekt würde ich nicht einfach loslegen, sondern zuerst die Anforderungen vollständig erfassen. Dazu gehört, die Vorgaben beider Fächer vor Sprint 1 einzuholen und für jede Funktion aufzuschreiben, was der Nutzer erwartet. Die Tests würde ich aus diesem erwarteten Verhalten ableiten und nicht aus dem fertigen Code, dann hätte ich den Fehler bei den Filmvorschlägen selbst gefunden. Die Schichten im Backend würde ich von Anfang an anlegen,

und Annahmen zur Sicherheit würde ich mit einem Test prüfen, der den Angriff nachstellt. Außerdem sehe ich mir vor jedem Commit `git status` und `git diff` an und vergleiche nach jeder Änderung auf Heroku die Werte beider Apps.

Dem nächsten Jahrgang würde ich empfehlen, so früh wie möglich anzufangen, denn das Projekt ist auch mit KI sehr aufwendig. Außerdem lohnt es sich, ein Team zu suchen, in dem jeder gleich viel macht und die Chemie stimmt. Gute Absprachen haben uns mehr Zeit gespart als jedes Werkzeug. KI ist dabei eine große Hilfe, aber nur, wenn man jeden Vorschlag selbst prüft und den Code danach ohne KI erklären kann.

4.2 Wissenschaftliche Reflexion – Madlen Hahn

Strukturiertes und organisiertes Arbeiten

Bei meiner Arbeit im Projekt CineRate habe ich mich an den Sprints orientiert. Dort wurden beispielsweise die umzusetzenden User Stories festgelegt, sodass ich eigentlich immer wusste, was umgesetzt werden sollte und worauf der Fokus im jeweiligen Sprint lag. Mein Aufgabenbereich war das Backend-for-Frontend (BFF), also die Schnittstelle zwischen den beiden Oberflächen (SPA und MPA) und dem Backend.

Sehr hilfreich waren unsere regelmäßigen Teammeetings. Im Review haben wir gemeinsam besprochen, welche Aufgaben bereits abgeschlossen sind, woran wir noch arbeiten müssen und ob neue Ideen aufgenommen und umgesetzt werden sollen. Anschließend haben wir festgelegt, welche Aufgaben im nächsten Sprint anstehen.

Geholfen haben mir auch die Pull Requests auf GitHub. Dadurch konnte ich sehen, woran die anderen gerade arbeiten, und mich an ihrem aktuellen Stand orientieren. Wenn beispielsweise im Backend eine neue Route fertiggestellt wurde, konnte ich nachvollziehen, welche Daten ich im BFF weitergeben muss. Dadurch hat die Zusammenarbeit aus meiner Sicht gut funktioniert. Meine Vorgehensweise bestand darin, mich regelmäßig über die Pull Requests der anderen auf dem Laufenden zu halten und die Sprintziele für meinen eigenen Aufgabenbereich umzusetzen.

Die Funktionen habe ich meistens Schritt für Schritt umgesetzt. Dabei bin ich beispielsweise mit der Kontoerstellung und dem Löschen von Konten gestartet und habe mich anschließend über die Filmsuche und Detailansicht bis hin zu Bewertungen, Watchlist, Filmvorschlägen und Kommentaren vorgearbeitet. Diese Reihenfolge habe ich gewählt, weil die Funktionen aufei-

nander aufbauen. Ohne Anmeldung gibt es kein Token, und ohne Token kann man weder bewerten noch etwas vormerken. So musste ich nie an mehreren Stellen gleichzeitig arbeiten. Für jede Funktion habe ich einen eigenen Branch und einen eigenen Pull Request erstellt. Bevor Änderungen in den Hauptzweig übernommen wurden, habe ich sie mithilfe unserer automatischen Tests überprüft. Diese wurden bei jedem Pull Request über GitHub Actions ausgeführt.

Stärken und Schwächen

Die größte Schwierigkeit war für mich am Anfang, dass ich bisher noch keine vergleichbare Aufgabe umgesetzt hatte. Ich war mir unsicher, womit ich anfangen sollte und wie ich die einzelnen Funktionen umsetzen könnte. Mit GitHub hatten wir zwar bereits gearbeitet, allerdings noch nicht mit fünf verschiedenen Repositories und Pull Requests. Deshalb musste ich mich zunächst in diese Arbeitsweise einarbeiten. Ungewohnt war das vor allem, weil eine Änderung in einem Repository oft auch eine Anpassung in einem anderen nach sich gezogen hat. Auch der Unterschied zwischen REST und GraphQL war mir anfangs nicht richtig klar. Erst während der Umsetzung habe ich besser verstanden, warum unser BFF beide Schnittstellen anbietet.

Da ich mir zu Beginn noch unsicher war, wie das Zusammenspiel mit dem tatsächlichen Backend funktioniert, habe ich meine Routen zunächst nur mit einem simulierten Backend getestet. Weil diese Tests bestanden haben, hatte ich eine Sicherheit, die nicht ganz gestimmt hat, denn sie prüfen nur, ob das BFF mit vorgegebenen Antworten richtig umgeht. Erst später habe ich die Routen auch gegen das echte Backend auf Heroku geprüft. Dabei ist mir aufgefallen, dass manche Probleme erst bei der tatsächlichen Einbindung sichtbar werden, zum Beispiel ob eine neue Route überhaupt richtig eingebunden ist.

Als Stärke sehe ich, dass ich meine Aufgaben zuverlässig umgesetzt und mich regelmäßig mit meinem Team abgestimmt habe. Auch wenn ich mich anfangs erst in viele neue Dinge einarbeiten musste, konnte ich meine Aufgaben nach und nach umsetzen.

Was ich beim nächsten Mal anders machen würde

Beim nächsten Projekt würde ich vieles genauso machen wie bisher. Besonders froh bin ich darüber, dass wir uns im Team regelmäßig ausgetauscht und früh genug mit der Umsetzung begonnen haben. Das würde ich auf jeden Fall beibehalten. Auch die Funktionen nacheinander umzusetzen, hat mir geholfen, den Überblick zu behalten.

Anders machen würde ich vor allem, von Anfang an auch gegen das echte Backend zu testen und mich nicht zunächst nur auf ein simuliertes Backend zu verlassen. Außerdem würde ich

mir zu Beginn mehr Zeit nehmen, um das Zusammenspiel der einzelnen Systemteile besser zu verstehen, bevor ich mit der eigentlichen Umsetzung anfange. So hätte ich manche Schwierigkeiten früher bemerkt.

Empfehlung für Studierende des nächsten Jahres

Ich würde den Studierenden im nächsten Jahr raten, sich innerhalb der Gruppe regelmäßig abzusprechen, gemeinsame Ziele festzulegen und die Aufgaben klar zuzuteilen. So behält jeder den Überblick darüber, was bereits umgesetzt wurde, welche Aufgaben noch offen sind und wer dafür zuständig ist.

4.3 Wissenschaftliche Reflexion – Nico Weber

Reflexion der eigenen Arbeit

Die Rolle des Product Owners hat aus meiner Sicht gut zu mir gepasst. Ich beschäftige mich viel mit Filmen und bin oft auf Seiten wie IMDB unterwegs. Deshalb hatte ich von Anfang an einen guten Überblick darüber, welche Anforderungen für CineRate am wichtigsten sind, wie die Oberfläche aussehen sollte und auf welche Funktionen man notfalls verzichten kann. Als Product Owner habe ich außerdem das Product Backlog, das Sprint Backlog und die Sprint Reviews geführt. Das hat im Rückblick sehr gut funktioniert, weil wir die Items von Beginn an sinnvoll auf die Sprints aufteilen konnten und so alle Muss-Anforderungen innerhalb von drei Sprints umgesetzt haben. Die gemeinsam im Team durchgeführten Sprint Reviews halfen dabei, die verbleibende Zeit und den Stand der Items im Blick zu behalten.

Als Entwickler der SPA und der MPA konnte ich einiges aus der Vorlesung Fortgeschrittene Systementwicklung in HTML, CSS und JavaScript anwenden. Einzelne Dateien waren dennoch anspruchsvoll und zeitaufwendig, weshalb an entsprechend gekennzeichneten Stellen im Code und dem KI-Log der Code mit Unterstützung von KI erstellt wurde. Des Weiteren musste ich teilweise warten, bis der passende Code im Backend und im BFF fertig war, um nicht gegen ein Mockup zu entwickeln. In der Regel wurde jedoch bereits im Voraus im Team geklärt, welche Funktion mit welchen Anforderungen als Nächstes umgesetzt wird, sodass ich mich darauf vorbereiten konnte.

Manche technischen Entscheidungen, Architekturentscheidungen und Entscheidungen im Code sind mir als Product Owner dennoch schwergefallen, da mein Wissen dafür nicht tief genug war. In solchen Fällen haben wir uns im Team zusammengesetzt, uns informiert und gemeinsam entschieden. Drei dieser Entscheidungen möchte ich begründen. Erstens haben wir die SPA mit Vanilla-JavaScript umgesetzt und auf ein Framework verzichtet. Die Grundlagen kannten wir aus der Vorlesung, und bei der knappen Projektzeit wollten wir uns nicht noch in ein Framework einarbeiten. Zudem blieb der Code für alle nachvollziehbar. Nachteilig ist jedoch der Mehraufwand, etwa beim Routing. Zweitens speichern wir das Token nach der Anmeldung im localStorage des Browsers und senden es bei jeder Anfrage mit. Das ist einfach umzusetzen und funktioniert auch ohne Cookie-Konfiguration, obwohl SPA, BFF und Backend als getrennte Anwendungen unter eigenen Adressen laufen. Die dritte Entscheidung betrifft den Umfang. CineRate beschränkt sich aus zeitlichen Gründen ausschließlich auf Filme. Serien werden bewusst nicht behandelt, da dies im Rahmen der vorgegebenen Zeit nicht realisierbar wäre.

Struktur und Organisation

Meines Erachtens haben wir strukturiert und organisiert gearbeitet. Die Aufgaben waren dabei klar verteilt: Lukas war für das Backend und die Datenbank zuständig, Madlen für den BFF und ich für die SPA und die MPA. Dadurch konnte sich jeder auf seinen Bereich konzentrieren und es gab für jede Komponente einen festen Ansprechpartner. An den Stellen, an denen die Komponenten zusammenarbeiten, haben wir uns im Team abgestimmt. Neben Backlog und Sprint Reviews haben wir in GitHub auf eigenen Branches entwickelt und die Änderungen anschließend auf main übernommen. Die automatisierten Tests mit GitHub Actions haben dabei gezeigt, ob bestehende Funktionen nach einer Änderung noch funktionieren. Auch die Bereitstellung der Daten über REST und GraphQL hat gut funktioniert. Die SPA verwendet REST, die MPA bezieht die Filmsuche über GraphQL, und beide greifen über denselben BFF zu.

Stärken und Schwächen

Zu meinen Stärken zähle ich die umfangreiche Testabdeckung und das saubere Fehlerhandling in SPA und MPA. Außerdem habe ich meine Aufgaben gewissenhaft umgesetzt, und die Kommunikation im Team war insgesamt, auch von meiner Seite, stets gut.

Schwächen sehe ich an drei Stellen. Zu Beginn hat es Zeit gekostet, den Überblick über das Gesamtprojekt zu bekommen, auch weil wir die Vorgaben der Vorlesung Fallstudie mitberücksichtigen mussten. Wir hätten die Vorgaben beider Fächer früher gemeinsam durchgehen können. Außerdem war ich beim Schreiben von Code teilweise von KI abhängig, da mein eigenes Wissen in einigen Fällen nicht ausgereicht hat. Ich habe versucht, das mit Tests und der Kennzeichnung im KI-Log auszugleichen, die Abhängigkeit bleibt aber bestehen. Die dritte Schwäche ist, dass ich zu spät mit der MPA begonnen habe, weil lange unklar war, welche Funktionen sie enthalten muss.

Was ich anders machen würde

In einem ähnlichen Projekt würde ich früher mit der MPA und mit GraphQL beginnen. Es hat viel Zeit gekostet, mich in beide Themen einzudenken, sie zu verstehen und im Code umzusetzen. Außerdem würde ich die Dokumentation für Webprogrammierung und Fallstudie sowie die Grafiken und Modelle parallel zur Entwicklung der Webanwendung erstellen und nicht erst, wenn die Webanwendung fertig ist, sodass die Dokus nicht kurz vor der Abgabe von Grund auf neu erstellt werden müssen. Das sind auch die Punkte, die ich den zukünftigen Studierenden empfehlen würde, die im dritten Semester ein ähnlichen Projekt umsetzen werden.

Anhang

ADR-01: Architekturstil – Schichtenarchitektur mit Backend-for-Frontend

- **Status:** vom Team bestätigt am 27.10.2026; Teilentscheidung „Zugriff nur über das BFF“ ergänzt am 06.11.2026 (Lukas, Team-Bestätigung ausstehend), auf Heroku aktiv seit 07.11.2026; MPA und GraphQL nachgetragen am 07.11.2026 (Stand der Umsetzung, beschlossen in ADR-07 und am 13./14.10.2026)
- **Sprint:** 1
- **Betrifft:** alle Komponenten (`cinerate-spa`, `cinerate-mpa`, `cinerate-bff`, `cinerate-backend`, `cinerate-db`)

Kontext

CineRate ist eine Web-Anwendung zum Verwalten und Bewerten von Filmen. Filmdaten stammen aus der externen TMDB-API, Nutzer-, Bewertungs- und Watchlist-Daten liegen in einer eigenen Datenbank. Randbedingungen:

- Laut Aufgabenstellung des Dozenten sind getrennte Repositories mit BFF-Muster vorgesehen.
- Das Team hat keine Erfahrung mit Frameworks. Eingesetzt werden HTML, CSS und Vanilla-JavaScript im Frontend sowie Node.js mit Express im Backend.
- Deployment auf Heroku, drei Apps (SPA, BFF, Backend), PostgreSQL am Backend. Seit Sprint 2 kommt die MPA als vierte App dazu (ADR-07).
- Das Team arbeitet zu dritt. Die Komponenten sollen parallel entwickelt und getrennt getestet werden können.

Betrachtete Alternativen

Alternative	Vorteile	Nachteile
A: Monolith (eine Express-App liefert Oberfläche, Logik und Datenzugriff)	Ein Repository, ein Deployment, kein zusätzlicher Netzwerkaufruf	Keine Trennung der Zuständigkeiten, Oberfläche und Logik nur gemeinsam änderbar, widerspricht der Vorgabe getrennter Repositories
B: Zwei Schichten (SPA ruft Backend direkt auf, TMDB-Aufrufe in der SPA)	Weniger Komponenten, kein BFF	Die SPA kennt die Backend-Schnittstelle und die TMDB-Schnittstelle, API-Keys wären im Browser sichtbar, Daten aus mehreren Quellen müsste die SPA selbst zusammenführen
C: Schichtenarchitektur mit BFF (SPA → BFF → Backend → Datenbank)	Klare Trennung, SPA kennt nur eine Schnittstelle, externe Zugriffe und Schlüssel bleiben serverseitig, Komponenten getrennt testbar	Mehr Repositories und Deployments, jede Anfrage durchläuft eine Station mehr

Entscheidung

Wir wählen **C**, eine Schichtenarchitektur mit vier Schichten:

1. **Präsentationsschicht:** zwei Touchpoints, `cinerate-spa` (Single-Page-App im Browser) und `cinerate-mpa` (Multi-Page-App, Seiten vom Server, ADR-07). Beide sprechen ausschließlich mit dem BFF.
2. **Vermittlungsschicht:** `cinerate-bff` (Präfix `/bff`). Einzige Schnittstelle von SPA und MPA, leitet Anfragen an das Backend weiter. Es bietet die Daten über REST und zusätzlich über GraphQL an (`/bff/graphql`, siehe Teilentscheidung). Das BFF ruft die TMDB-API **nicht** selbst auf und hat keinen Datenbankzugriff.
3. **Logikschicht:** `cinerate-backend` (Präfix `/api`). REST-API mit Fachlogik für Nutzer, Anmeldung, Filme und Bewertungen. Es ruft bei Bedarf die TMDB-API auf und speichert die Filme in der Datenbank (siehe Teilentscheidung). Nur das BFF darf es aufrufen (gemeinsamer Schlüssel, siehe Teilentscheidung vom 06.11.2026).
4. **Datenschicht:** PostgreSQL, Schema und Migrationen in `cinerate-db`.

Begründung: Die Variante erfüllt die Vorgabe des Dozenten, hält Zugangsdaten und Schlüssel aus dem Browser heraus und erlaubt es, die Komponenten unabhängig zu entwickeln.

Teilentscheidung: Wer ruft die TMDB-API auf? (Team, 09.10.2026)

Betrachtet wurden zwei Varianten:

Variante	Vorteile	Nachteile
A: BFF ruft TMDB auf und schickt die Daten zum Speichern an das Backend	Passt zur Beschreibung des BFF-Musters (Zugriff auf mehrere Datenquellen, Kombination der Daten)	Das Backend braucht eine zusätzliche Route zum Speichern von Filmen (nicht im API-Design). Das BFF bräuchte Wissen über Cache und Speicherung. Bei den Empfehlungen holt das BFF Bewertungen vom Backend, berechnet das Lieblingsgenre und speichert danach wieder über das Backend.
B: Backend ruft TMDB auf, speichert die Filme in der Datenbank und antwortet	Bewertungen, Lieblingsgenre, Cache und TMDB-Aufruf liegen an einer Stelle. Passt zum API-Design, zu den abgegebenen Use Cases und zum Beschluss vom 02.10.	Das BFF reicht für Filmdaten nur durch. Der vorhandene TMDB-Client im BFF muss ins Backend verschoben werden.

Gewählt: B. Das Backend ruft die TMDB-API auf. Es prüft zuerst die Datenbank. Fehlt ein Film, holt es ihn bei TMDB, speichert ihn und antwortet. Das BFF reicht Anfragen durch. Der TMDB-Schlüssel liegt nur im Backend (Heroku-Umgebungsvariable).

Teilentscheidung: Wo liegt GraphQL? (Team, 13.10.2026; Rückmeldung Herr Hartwig, 14.10.2026)

Die Daten sollen über REST und über GraphQL angeboten werden.

Variante	Vorteile	Nachteile
A: GraphQL im BFF, Resolver rufen das Backend per REST auf	Keine weitere App, Fachregeln bleiben nur im Backend, SPA und MPA kennen weiterhin nur das BFF	REST und GraphQL liegen in einem Server; jede API-Änderung muss in Backend-Route, REST-Route des BFF, Resolver und Schema nachgezogen werden
B: Eigener GraphQL-Server neben dem BFF	Getrennte Entwicklungswege, wie in den Beispielprojekten	Weitere App auf Heroku, zweiter Einstiegspunkt für die Touchpoints
C: GraphQL direkt auf der Datenbank	Keine Umwege über REST	Fachregeln lägen doppelt vor, die Datenbank wäre nicht mehr nur über das Backend erreichbar

Gewählt: A (Pakete `graphql` und `graphql-http`, Node.js). Herr Hartwig hat am 14.10.2026 geantwortet, dass REST und GraphQL normalerweise getrennte Server wären, er die Lösung im BFF aber akzeptiert. Stand 07.11.2026: Das Schema bietet sieben Abfragen und zehn Mutationen und deckt damit alle REST-Routen ab. Die MPA nutzt GraphQL für Filmsuche und Filmdetail, die SPA nutzt REST.

Teilentscheidung: Wie stellen wir sicher, dass nur das BFF das Backend aufruft? (Lukas, 06.11.2026)

Bisher sollte CORS dafür sorgen (`BFF_URL`). Beim Code-Review am 06.11.2026 fiel auf, dass das nicht funktioniert: CORS ist eine Regel, die nur Browser einhalten. Das BFF ruft das Backend von Server zu Server auf, dort greift CORS nicht. Jeder, der die Heroku-Adresse kennt, konnte das Backend mit `curl` oder Postman direkt aufrufen. War `BFF_URL` leer, waren sogar alle Origins erlaubt (`*`).

Variante	Vorteile	Nachteile
A: CORS beibehalten	Kein Umbau	Schützt nicht gegen Aufrufe außerhalb des Browsers, also keine echte Einschränkung
B: Gemeinsamer Schlüssel im Header (`X-Internal-Key`, Wert in der Config Var `INTERNAL_API_KEY` auf beiden Heroku-Apps)	Einfach, ohne zusätzliche Bibliothek, wirkt gegen jeden Aufrufer ohne Schlüssel	Schlüssel muss sicher verwaltet und auf zwei Apps gleich gesetzt werden; das BFF muss ihn bei jedem Aufruf mitschicken
C: Netzwerk-Isolation (Backend nur im privaten Netz erreichbar, z. B. Heroku Private Spaces)	Backend ist von außen gar nicht erreichbar	Kostenpflichtig, für das Projekt zu aufwendig

Gewählt: B. Ohne oder mit falschem Schlüssel antwortet das Backend mit 403 „Access is only allowed via the BFF." (403 statt 401, weil die SPA bei 401 zur Anmeldung springt). Der Vergleich läuft in konstanter Zeit (SHA-256 und `crypto.timingSafeEqual`). `GET /api/status` bleibt ohne Schlüssel erreichbar. Ist `INTERNAL_API_KEY` nicht gesetzt, prüft das Backend nicht; so konnte das Backend vor dem BFF ausgerollt werden. Scharf geschaltet wird durch Setzen der Config Var auf dem Backend, nachdem das BFF den Header mitschickt. Das ist seit dem 07.11.2026 der Fall; beim Ausrollen stand im Backend zuerst ein falscher Wert, und jede Anfrage des BFF wurde mit 403 abgelehnt. CORS wurde aus dem Backend entfernt.

Konsequenzen

- Einfacher: getrennte Entwicklung und Tests je Schicht, ein stabiler Vertrag zwischen SPA und BFF.
- Schwieriger: fünf Repositories, vier Apps auf Heroku, zusätzlicher Aufruf pro Anfrage (Antwortzeit), Änderungen an der API müssen in BFF (REST und GraphQL) und Backend nachgezogen werden.
- Durch die TMDB-Entscheidung: Der TMDB-Client wird vom BFF ins Backend verschoben, der TMDB-Schlüssel gehört in die Umgebungsvariablen des Backends. Das BFF ist für Filmdaten ein reiner Durchreicher.
- Durch die Teilentscheidung vom 06.11.2026: Ein geheimer Schlüssel mehr in der Konfiguration (BFF und Backend). Wer das Backend direkt testet (Postman, `curl` gegen Heroku), muss den Header mitschicken. Umsetzung: `middleware/requireInternalKey.js` im Backend (KI-Log A45), Header in allen Backend-Aufrufen des BFF (`lib/backend.js`).
- Betroffene Artefakte: Architekturdiagramm (TMDB hängt am Backend; Pfeil BFF → Backend mit `X-Internal-Key`), Klassendiagramm (Zuordnung der Klassen zu Schichten), Sequenzdiagramme (Beteiligte: SPA, BFF, Backend, Datenbank, bei Filmen zusätzlich TMDB), READMEs von `cinerate-bff` und `cinerate-backend`, `CineRate-Use-Cases.pdf` und `CineRate_Akteure-1.pdf` (Wortwahl), ADR-02.

ADR-02: Controller für die Systemoperationen von US-2

- **Status:** vom Team bestätigt am 27.10.2026
- **Sprint:** 1
- **Betrifft:** `cinerate-backend` (`routes/users.js`, `routes/auth.js`), `cinerate-bff`
- **GRASP-Pattern:** Controller, High Cohesion

Kontext

Unterhalb der Oberfläche muss ein Objekt die Systemereignisse empfangen und die Abläufe koordinieren. Bei US-2 sind das die Systemoperationen Registrieren, Anmelden, Abmelden und Account löschen. Die Oberfläche (SPA) sendet sie als HTTP-Anfragen an den BFF, der sie an das Backend weiterleitet. Das Controller-Objekt soll selbst wenig Fachlogik enthalten und nur koordinieren.

Rahmen: ADR-01 (Schichtenarchitektur), Node.js mit Express ohne weiteres Framework.

Betrachtete Alternativen

Alternative	Vorteile	Nachteile
A: Der BFF übernimmt die Koordination (BFF prüft, ruft die Datenbank auf, erzeugt Tokens)	Eine einzige Eintrittsstelle	Der BFF bekäme Fachlogik und Datenbankzugriff, das widerspricht der Aufgabe der Schicht, die Logik gehört ins Backend
B: Ein zentraler Controller für alle Systemoperationen (Fassaden-Controller)	Einfach zu finden	Wächst mit jeder Story, geringe Kohäsion, schwer zu testen
C: Ein Controller je Anwendungsbereich im Backend (Router-Modul `users` für Registrieren und Account löschen, Router-Modul `auth` für Anmelden und Abmelden); der BFF leitet nur durch	Hohe Kohäsion, jedes Modul hat eine klare Aufgabe, neue Bereiche kommen als neues Modul dazu	Mehrere Einstiegspunkte, Koordination über Bereichsgrenzen hinweg (z. B. Token-Prüfung) braucht eine gemeinsame Middleware

Entscheidung

Wir wählen **C** (Controller-Pattern nach GRASP, Use-Case-Controller auf Bereichsebene):

- `routes/users.js` ist der Controller für Registrieren (`POST /users`) und Account löschen (`DELETE /users/{userId}`).
- `routes/auth.js` ist der Controller für Anmelden (`POST /auth`) und Abmelden (`DELETE /auth`).
- Die Controller prüfen die Eingabe nicht selbst, hashen keine Passwörter und erzeugen keine Tokens. Sie rufen die zuständigen Module auf und setzen die HTTP-Antwort (siehe ADR-03).
- Der BFF ist für US-2 **kein Controller**: Er reicht Methode, Body und `Authorization`-Header unverändert weiter. Seit dem 06.11.2026 schickt er zusätzlich den Header `X-Internal-Key` mit (ADR-01).

Begründung: Jeder Controller hat eine kleine, zusammengehörige Aufgabe (High Cohesion). Die Fachlogik bleibt im Backend.

Zusammenhang mit ADR-01: Das Backend ruft auch die TMDB-API auf (Teilentscheidung vom 09.10.2026). Damit koordiniert auch bei Filmdaten und Empfehlungen (US-1) das Backend, und der BFF bleibt ein reiner Durchreicher. Die Controller für diese Stories entstehen später als weitere Router-Module im Backend nach demselben Muster.

Konsequenzen

- Einfacher: neue Systemoperationen kommen als Route im passenden Modul oder als neues Modul dazu. Die Controller sind klein.
- Schwieriger: Querschnittsfunktionen wie die Token-Prüfung brauchen eine gemeinsame Middleware (`requireAuth`).
- Betroffene Artefakte: Klassendiagramm (Controller `users`, `auth`), Sequenzdiagramme zu Registrieren und Anmelden (erste Nachricht nach dem BFF geht an den Controller), ADR-01, ADR-03.

ADR-03: Fachlogik in eigenen Modulen (Low Coupling, High Cohesion, Information Expert)

- **Status:** vom Team bestätigt am 27.10.2026; die Entscheidung gegen eine Service-Schicht (Alternative B) ist durch ADR-05 ersetzt, siehe Hinweis unten
- **Sprint:** 1
- **Betrifft:** `cinerate-backend` (`lib/validation.js`, `lib/password.js`, `lib/tokens.js`, `middleware/requireAuth.js`)
- **GRASP-Pattern:** Low Coupling, High Cohesion, Information Expert

  > **Hinweis (27.10.2026): Teilweise ersetzt durch ADR-05.** Diese Entscheidung hat in Sprint 1 eine Service-Schicht abgelehnt (Alternative B) und die Datenbankzugriffe in den Controllern belassen. Mit ADR-05 (bestätigt am 15.10.2026) hat das Backend die Schichten Route, Service und Repository erhalten. Seitdem gilt:
  >
  > - Die Datenbankzugriffe liegen in Repositories (`db/`), nicht mehr in den Routen. Die Routen sind Controller und koordinieren nur.
  > - Die Regeln und Abläufe liegen in Services (`services/`). Sie sind Information Expert für ihre Fachregeln und rufen Repositories und den TMDB-Client auf.
  > - Unverändert gültig bleiben die kleinen Module in `lib/` (`validation`, `password`, `tokens`) mit **High Cohesion** und **Low Coupling**. Die Services bekommen ihre Abhängigkeiten über Fabrikfunktionen übergeben und lassen sich so mit Ersatzobjekten ohne Datenbank testen (Low Coupling).
  >
  > Für die Sequenzdiagramme gilt der Stand nach ADR-05.

Kontext

Bei US-2 fallen mehrere Teilaufgaben an: Eingaben prüfen, Passwörter sicher speichern und vergleichen, Tokens erzeugen und prüfen, mit der Datenbank sprechen. Es muss entschieden werden, welches Objekt welche Verantwortung trägt. Zusätzlich fordert die Definition of Done automatisierte Unit-Tests für die Kernlogik. Tests ohne Datenbank und ohne HTTP-Server sind dafür deutlich einfacher.

Rahmen: ADR-02 (Controller), Express ohne weiteres Framework, kein Klassenkonzept in der Codebasis (Module mit Funktionen).

Betrachtete Alternativen

Alternative	Vorteile	Nachteile
A: Alles im Controller (Prüfung, Hash, Token und Datenbankzugriff direkt in den Route-Funktionen)	Wenig Dateien, auf den ersten Blick einfach	Controller sind lang und unübersichtlich, geringe Kohäsion, Tests brauchen Datenbank und Server
B: Zusätzliche Service-Schicht (z. B. `UserService`, `AuthService` mit Datenbankzugriff)	Klare Schichten, in größeren Projekten üblich	Mehr Struktur als bei dieser Größe nötig, zusätzliche Abstraktion ohne Framework-Unterstützung
C: Kleine Module je Verantwortung (Prüfung, Passwort, Token) plus Middleware für die Zugriffsprüfung; Datenbankzugriffe bleiben im Controller	Hohe Kohäsion, geringe Kopplung, Module sind ohne Datenbank testbar	Mehrere kleine Dateien, Datenbankzugriff weiterhin in den Controllern

Entscheidung

Wir wählen **C**:

- **High Cohesion:** Jedes Modul hat genau eine Aufgabe. `validation.js` prüft Eingaben (Registrierung, E-Mail, Passwort mindestens 8 Zeichen). `password.js` erzeugt und vergleicht bcrypt-Hashes. `tokens.js` erzeugt Tokens und deren SHA-256-Hash.
- **Low Coupling:** Die Module sind reine Funktionen ohne Datenbank- und HTTP-Bezug. Die Controller (ADR-02) hängen nur von deren Schnittstelle ab, nicht von ihrer Umsetzung.
- **Information Expert:** Wer die Information hat, übernimmt die Aufgabe. Die Regeln zur Eingabeprüfung kennt `validation.js`. Wie ein Token gehasht wird, weiß `tokens.js`. Ob ein mitgeschicktes Token zu einer gültigen Sitzung gehört, entscheidet `requireAuth` anhand der Tabelle `Sitzung` (siehe ADR-04).

Begründung: Die Aufteilung macht die Kernlogik einzeln testbar und hält die Controller klein. Die Service-Schicht aus B wäre bei dieser Größe zusätzlicher Aufwand ohne klaren Nutzen.

Konsequenzen

- Einfacher: Unit-Tests ohne Datenbank (28 Tests, 100 % Testabdeckung in `lib/`). Änderungen an Regeln, etwa der Passwortlänge, betreffen nur ein Modul.
- Schwieriger: Mehr Dateien. Die Datenbankzugriffe in den Controllern werden nicht automatisiert getestet, sondern manuell gegen die Datenbank.
- Betroffene Artefakte: Klassendiagramm (Module und ihre Abhängigkeiten), Sequenzdiagramm zu Registrieren und Anmelden (Aufrufe von Controller an `validation`, `password`, `tokens` sind dort mit den GRASP-Patterns zu kennzeichnen), ADR-02, ADR-04.

ADR-04: Anmeldung mit zufälligem Token statt JWT

- **Status:** entschieden; ergänzt am 06.11.2026 (Aufräumen abgelaufener Sitzungen, Begrenzung der Anmeldeversuche; Lukas, Team-Bestätigung ausstehend)
- **Datum:** 09.10.2026 (Entscheidung am 06.10.2026, Token-Laufzeit am 07.10.2026 bestätigt)
- **Beteiligte:** Nico Weber, Madlen Hahn, Lukas Krebelder
- **Betrifft:** User Story US-2 (Registrieren, Anmelden, Abmelden, Account löschen), `cinerate-backend`, `cinerate-db`

Kontext

Nach der Anmeldung muss das System bei jeder geschützten Anfrage erkennen, welcher Nutzer sie sendet. Die API schickt dafür ein Token im Header `Authorization: Bearer <token>`. Die Anforderungen aus US-2:

- **Abmelden** muss das Token ungültig machen (Acceptance Criteria zu US-2, `DELETE /auth`).
- **Account löschen** muss alle Tokens des Nutzers ungültig machen.
- Die SPA spricht nur mit dem BFF. Das BFF reicht den Header unverändert an das Backend weiter. Geprüft wird im Backend.
- Das Team hat keine Vorerfahrung mit Frameworks oder Authentifizierungsbibliotheken. Die Lösung soll einfach und nachvollziehbar sein.

Im ersten Architekturentwurf war ein JWT (JSON Web Token) vorgesehen.

Entscheidung

Wir verwenden ein **zufälliges Token** mit serverseitiger Speicherung der Sitzung:

1. Bei `POST /auth` prüft das Backend E-Mail und Passwort (Passwort als bcrypt-Hash gespeichert).
2. Es erzeugt ein Token aus 32 zufälligen Bytes (64 Hex-Zeichen, `crypto.randomBytes`).
3. In der neuen Tabelle `Sitzung` wird **nur der SHA-256-Hash** des Tokens gespeichert, zusammen mit `nutzer_id`, `erstellt_am` und `laeuft_ab_am`. Das Token selbst bekommt nur der Nutzer.
4. Ein Token ist **24 Stunden** gültig (Standardwert in der Datenbank).
5. Bei jeder geschützten Route (`middleware/requireAuth.js`) hasht das Backend das mitgeschickte Token und sucht es in `Sitzung` (`token_hash` passt, `laeuft_ab_am` liegt in der Zukunft). Sonst antwortet es mit 401 "Invalid token."
6. `DELETE /auth` löscht den Eintrag der aktuellen Sitzung. Beim Löschen eines Accounts werden die Einträge über `ON DELETE CASCADE` mitgelöscht.

Ergänzung vom 06.11.2026:

7. Bei jeder erfolgreichen Anmeldung löscht das Backend alle abgelaufenen Einträge aus `Sitzung` (`DELETE … WHERE laeuft_ab_am <= NOW()`, nutzt den Index `idx_sitzung_laeuft_ab`).
8. Nach **5 fehlgeschlagenen Anmeldungen** mit derselben E-Mail ist die Anmeldung **15 Minuten** gesperrt: 429 „Too many failed login attempts. Please try again later.“, auch mit richtigem Passwort. Unbekannte E-Mails zählen ebenfalls, eine erfolgreiche Anmeldung setzt den Zähler zurück. Gezählt wird **pro E-Mail, nicht pro IP**: Alle Anfragen kommen über das BFF, das Backend sieht für alle Nutzer dieselbe IP. Der Zähler liegt im Arbeitsspeicher (`lib/attemptLimiter.js`).

SHA-256 statt bcrypt: Das Token hat 256 Bit Zufall und lässt sich nicht erraten. Ein schneller Hash genügt. Außerdem muss der Hash bei gleichem Token immer gleich sein, damit die Datenbank ihn direkt finden kann (bcrypt erzeugt bei jedem Aufruf einen anderen Hash).

Betrachtete Alternativen

Alternative	Vorteile	Gründe gegen die Wahl
JWT (signiertes Token ohne Speicherung)	Keine Datenbankabfrage pro Anfrage	Ein JWT bleibt bis zum Ablauf gültig. Abmelden und Account löschen würden das Token nicht sofort ungültig machen, dafür wäre zusätzlich eine Sperrliste in der Datenbank nötig. Damit entfällt der Vorteil. Zusätzliche Bibliothek und Schlüsselverwaltung.
Zufälliges Token im Klartext speichern	Noch einfacher	Bei einem Datenbankleck wären alle gespeicherten Tokens direkt nutzbar.
E-Mail und Passwort bei jeder Anfrage senden (Basic Auth)	Keine Sitzungen	Das Passwort würde bei jeder Anfrage übertragen. Abmelden wäre nicht möglich.

Folgen

Vorteile

- Abmelden und Account löschen wirken sofort.
- Bei einem Datenbankleck lassen sich aus den gespeicherten Hashes keine gültigen Tokens ableiten.
- Keine zusätzliche Bibliothek, nur `crypto` aus Node.js.
- Mehrere Anmeldungen (mehrere Geräte) pro Nutzer sind möglich, weil jede Anmeldung einen eigenen Eintrag erzeugt.

Nachteile und offene Punkte

- Jede geschützte Anfrage braucht eine Datenbankabfrage.
- ~~Abgelaufene Einträge bleiben in `Sitzung` stehen.~~ Seit 06.11.2026 erledigt: Sie werden bei jeder erfolgreichen Anmeldung gelöscht.
- Die Login-Sperre zählt im Arbeitsspeicher: Nach einem Neustart des Backends beginnt der Zähler bei 0, bei mehreren Dynos hätte jeder seinen eigenen. Für einen Dyno reicht das.
- Wer die E-Mail eines anderen kennt, kann dessen Anmeldung durch 5 falsche Versuche für 15 Minuten sperren. Bewusst in Kauf genommen (kurze Sperrzeit) gegenüber unbegrenztem Raten von Passwörtern.
- Die Touchpoints müssen das Token speichern und bei 401 zur Anmeldung weiterleiten. Umgesetzt: Die SPA speichert es im `localStorage` (Schlüssel `cinerate_token`), ein zentraler Handler in `api.js` meldet bei 401 auf einer geschützten Route ab und wechselt zur Anmeldung. Die MPA legt es serverseitig in ein `HttpOnly`-Cookie (ADR-07).
- Die 24 Stunden Laufzeit stehen noch nicht in `Acceptance-Criteria.pdf` (Annahme A6), das Dokument muss ergänzt werden.

Betroffene Artefakte: ER-Diagramm (Tabelle `Sitzung`, neue Spalten bei `Nutzer`), Sequenzdiagramm und Klassendiagramm zu US-2 (Anmelden, Abmelden), `Acceptance-Criteria.pdf` (Annahme A6), `CineRate_API-2.pdf`.

Umsetzung

`cinerate-backend`: `lib/tokens.js`, `middleware/requireAuth.js`, `routes/auth.js`, `routes/users.js`, seit dem Umbau nach ADR-05 zusätzlich `services/authService.js` (Anmelden, Abmelden) und `db/sessionRepository.js` (SQL auf `Sitzung`). `cinerate-db`: `sql/003_auth.sql`. Unit-Tests in `test/tokens.test.js`.

Ergänzung 06.11.2026 (KI-Log A46): `lib/attemptLimiter.js`, `services/authService.js` (Sperre), `db/sessionRepository.js` (`deleteExpiredSessions`). Tests in `test/attemptLimiter.test.js`, `test/authService.test.js`, `test/sessionRepository.test.js`.

ADR-05: Schichtenarchitektur im Backend – Route, Service, Repository

- **Status:** vom Team bestätigt am 15.10.2026 (Sprintplanung); Fehlerbehandlung angepasst am 06.11.2026 (Lukas, Team-Bestätigung ausstehend)
- **Datum:** 13.10.2026 (Vorschlag von Nico am 12.10.2026)
- **Beteiligte:** Nico Weber (Vorschlag), Lukas Krebelder (Entscheidung und Umsetzung), Madlen Hahn (Review)
- **Betrifft:** `cinerate-backend` (interne Struktur), Testkonzept, Portfolio (Backend, Codeaufbau)

Kontext

Im Backend stehen SQL, Fachregeln und HTTP-Behandlung gemischt in den Routen und in der Middleware. Nur `lib/` enthält reine Hilfsfunktionen (gut getestet). Der Stand vom 12.10.2026:

Datei	Aufgabe	SQL in der Datei
`routes/users.js`	Registrieren, Account löschen	`INSERT` und `DELETE` auf `Nutzer`
`routes/auth.js`	Anmelden, Abmelden	`SELECT` auf `Nutzer`, `INSERT` und `DELETE` auf `Sitzung`
`middleware/requireAuth.js`	Token prüfen	`SELECT` auf `Sitzung`
`routes/status.js`	Status	`SELECT COUNT` auf `Film`

Das sind sieben SQL-Abfragen in Routen und Middleware. Es kommen viele Funktionen dazu (Filme mit TMDB-Zwischenspeicher, Bewertungen, Watchlist, Kommentare, Empfehlungen). Alle brauchen den angemeldeten Nutzer und eigenes SQL. Weitere Randbedingungen:

- Eine Route mit SQL lässt sich nur mit Datenbank testen. Fachlogik in einem Service lässt sich mit einem Ersatz-Repository ohne Datenbank testen.
- Die Empfehlungen (US-1) kombinieren Bewertungen, Lieblingsgenre und TMDB. Diese Logik gehört nicht in eine Route.
- Das Backend zählt bei Web-Programmierung 15 Punkte, unter anderem für Codeaufbau und -strukturierung und die korrekte Umsetzung des API-Designs. Cache und Hintergrundverarbeitung (ADR-06) brauchen eine klare Stelle im Code.
- Das Team hat keine Vorerfahrung mit Frameworks. Es bleibt bei Node.js, Express und CommonJS.
- Die HTTP-API (Pfade, Statuscodes, Antworttexte) soll unverändert bleiben.

Entscheidung

Das Backend wird in drei Schichten gegliedert. Die Abhängigkeit geht nur in eine Richtung: Route → Service → Repository → `db/connection.js`.

Schicht	Ordner	Aufgabe	Darf nicht
Route	`routes/`	Anfrage lesen (`req.body`, `req.params`, `req.user`), Service aufrufen, Statuscode und JSON senden	Fachregeln oder SQL enthalten
Service	`services/`	Regeln und Abläufe: prüfen, Passwort hashen, Entscheidungen treffen, Repositories und TMDB-Client aufrufen	`req` / `res` kennen oder SQL enthalten
Repository	`db/`	SQL: lesen, schreiben, löschen	HTTP oder Fachregeln enthalten

Weitere Festlegungen:

1. **Services als Fabrikfunktionen**, z. B. `createUserService(userRepository)`. Das Repository lässt sich in Tests durch ein Ersatzobjekt austauschen.
2. **Repositories im Ordner `db/`** (neben `connection.js`), z. B. `userRepository.js`, `sessionRepository.js`, `movieRepository.js`.
3. **Fremdservices in `integrations/`** (ohne SQL). Der TMDB-Client zieht zusammen mit dem Cache-Modul (ADR-06) von `lib/` nach `integrations/` um.

4. **`lib/`** bleibt für reine Hilfsfunktionen (`validation`, `password`, `tokens`) und enthält neu `appError.js`.
5. **Fehler:** Fachliche Fehler werden als `AppError(status, message)` geworfen. ~~Die Route übersetzt sie mit `handleError(err, res)`~~. Seit 06.11.2026 übersetzt sie eine zentrale Fehler-Middleware (`errorMiddleware` in `lib/appError.js`, am Ende von `app.js`); Express 5 reicht Fehler aus async-Routen selbst dorthin weiter, die Routen haben kein `try/catch` mehr.
   - Der PostgreSQL-Fehler `23505` (UNIQUE verletzt) wird **im Repository** in `AppError(409)` übersetzt. Der Service kennt kein PostgreSQL.
   - ~~Alle anderen unerwarteten Fehler ergeben wie bisher **503** mit "Database is not reachable."~~ Seit 06.11.2026: **503** „Database is not reachable.“ nur, wenn die Datenbank nicht erreichbar ist (`isDatabaseUnavailable`: Netzwerkfehler, PostgreSQL-Verbindungsfehler), alle anderen unerwarteten Fehler **500** „Internal server error.“.
   - Auch Fehler vor den Routen kommen als JSON: ungültiges JSON im Body 400, zu großer Body 413, unbekannte Route 404 „Route not found.“ (vorher HTML-Seiten von Express).
6. **Umbau in kleinen Schritten**, jeweils ein eigener Pull Request mit Review. Dieselben Anfragen müssen dieselben Antworten liefern.

Schritt	Inhalt
1	`lib/appError.js`, `db/userRepository.js`, `services/userService.js`; `routes/users.js` nutzt sie (Vorbild). Neue Service-Tests.
2	Neue Film-Route (Suche mit Genrefilter) direkt in drei Schichten.
3	`db/sessionRepository.js`, `services/authService.js`; `requireAuth` nutzt das Session-Repository.
4	`routes/status.js`: SQL in `db/movieRepository.js` (z. B. `countMovies`). Umzug des TMDB-Clients nach `integrations/` zusammen mit dem Cache.

Die Umsetzung übernimmt Lukas als Besitzer des Backends. Madlen arbeitet am GraphQL im BFF.

Betrachtete Alternativen

Alternative	Vorteile	Gründe für oder gegen die Wahl
A: Alles in den Routen lassen	Kein Umbau	Dieselben Abfragen entstehen mehrfach, Fachlogik nur mit Datenbank testbar, Umbau später deutlich teurer. Nicht gewählt.
B: Route, Service, Repository	Klare Zuständigkeiten, Services ohne Datenbank testbar, klare Stelle für Cache und Hintergrundverarbeitung	**Gewählt.** Mehr Dateien.
C: Framework mit Schichtenstruktur (z. B. NestJS)	Fertige Struktur	Keine Vorerfahrung, zusätzlicher Lernaufwand und Abhängigkeit. Nicht gewählt.
Services als Klassen	Vertraute Schreibweise aus der objektorientierten Programmierung	Fabrikfunktionen passen besser zum bisherigen Funktionsstil in `lib/` und brauchen weniger Code. Nicht gewählt.
Ordner `repositories/` statt `db/`	Klarerer Name	`db/` besteht bereits, kein Umbenennen nötig. Nicht gewählt.

Folgen

Vorteile

- Services lassen sich wie `validation.js` ohne Datenbank testen (Ersatz-Repository). Das unterstützt die Qualitätssicherung (Unit-Tests der Kernlogik) und den Nachweis der Testabdeckung.
- Die Fachlogik der Empfehlungen (US-1) liegt in einem Service.
- Cache und Auslöser für Hintergrundverarbeitung (ADR-06) haben eine feste Stelle.

Nachteile und offene Punkte

- Mehr Dateien und etwas mehr Code je Funktion.
- Der Umbau kostet Zeit, die in der Sprintplanung berücksichtigt werden muss.

- ~~Unerwartete Fehler werden weiter als 503 gemeldet, auch wenn es kein Datenbankfehler ist.~~ Am 06.11.2026 behoben (500 für unerwartete Fehler, 503 nur bei Datenbankausfall). `CineRate_API-2.pdf` muss um 500 ergänzt werden.
- Das README von `cinerate-backend` muss die neue Struktur beschreiben.

Betroffene Artefakte: Klassendiagramm (Zuordnung der Klassen zu Schichten), Sequenzdiagramme, README von `cinerate-backend`, Testkonzept, ADR-06.

Umsetzung

Siehe Schritte oben. Grundlage ist der Entwurf `Vorschlag-Schichtenarchitektur-Backend.pdf` (Nico, 12.10.2026), der in einer Kopie von `origin/main` getestet wurde.

Anpassung der Fehlerbehandlung am 06.11.2026: PR #28 (zentrale Fehler-Middleware, `app.js` getrennt von `server.js`, KI-Log A42) und PR #29 (500 statt 503 für unerwartete Fehler, KI-Log A43).

ADR-06: Kommunikation zwischen den Komponenten – synchron, mit vorbereiteter asynchroner Verarbeitung

- **Status:** vom Team bestätigt am 15.10.2026 (Sprintplanung); Stand der Umsetzung nachgetragen am 07.11.2026 (Variante A umgesetzt, Variante B nicht gebaut)
- **Datum:** 13.10.2026
- **Beteiligte:** Lukas Krebelder (Entscheidung), Nico Weber und Madlen Hahn (zu bestätigen)
- **Betrifft:** `cinerate-backend` (TMDB-Anbindung, Cache), `cinerate-bff`, Portfolio Web-Programmierung

Kontext

Die Portfolio-Anforderungen von Herrn Hartwig (erhalten 12.10.2026) verlangen, dass die Kommunikation zwischen den Komponenten einer vorher definierten API folgt und **asynchron** ist. Begründete Ausnahmen sind erlaubt. Bei der Bewertung "Integration eines Fremdservices" (10 Punkte) zählen unter anderem saubere Verarbeitung, Fehlerbehandlung und Asynchronität.

Ist-Zustand: Alle Verbindungen zwischen den Komponenten sind **synchron** (Anfrage und Antwort):

Verbindung	Art
SPA → BFF, BFF → Backend	HTTP, REST (`/bff/...`, `/api/...`)
Backend → PostgreSQL	SQL über `pg`
Backend → TMDB	HTTP, GET

Im Code ist jeder Aufruf asynchron (`await`), der Node-Prozess blockiert also nicht. Zwischen den Komponenten gibt es keine Queue, keine Events und keine Hintergrundaufträge.

Weitere Randbedingungen:

- Die Nutzer brauchen das Ergebnis von Suche, Anmeldung und Bewertung sofort.
- Das Team ist klein, hat keine Vorerfahrung mit Frameworks und trägt die Kosten für Heroku selbst.
- Es ist unklar, ob Hartwig mit "asynchron" nur asynchrone Aufrufe im Code oder eine Entkopplung über Nachrichten und Queues meint. Das Team fragt ihn (vierte Frage in der Mail an Hartwig).

Entscheidung

Wir setzen zunächst **Variante A** um und bereiten **Variante B** vor:

1. **Synchron nur innerhalb der Backend-Komponenten.** BFF → Backend → Datenbank oder TMDB laufen als Anfrage und Antwort über HTTP bzw. SQL. Das ist die begründete Ausnahme, die Herr Hartwig ausdrücklich zulässt (siehe Rückmeldung unten). **Zwischen Frontend und BFF/Backend und zum Nutzer hin** gilt sie nicht: Die SPA ruft das BFF **asynchron und nicht blockierend** auf (`fetch` mit `async/await`, Ladeanzeige, Fehlerzustand; die Oberfläche bleibt bedienbar). Was länger dauern kann (Nachladen von Filmdaten), soll den Nutzer nicht warten lassen (siehe 3.).
2. **Asynchron im Code.** Alle externen Aufrufe laufen mit `async/await`. Mehrere voneinander unabhängige TMDB-Aufrufe (die Empfehlungen zu bis zu zehn Quellfilmen) laufen parallel mit `Promise.all`. Jeder TMDB-Aufruf hat ein Timeout. Bei Fehlern gibt es begrenzte Wiederholungen und klare Fehlerarten (Antwort 503 wie in `CineRate_API-2.pdf`).
3. **Vorbereitung auf Variante B** (Queue oder Hintergrundauftrag):
   - Die Arbeit "Film von TMDB holen und speichern" liegt in einer eigenen Service-Funktion (`importiereFilm(tmdbId)`). Sie kennt weder Route noch Queue.
   - Die Funktion ist **idempotent**: Speichern per Upsert (`INSERT … ON CONFLICT (tmdb_id) DO UPDATE`, `Film.tmdb_id` ist bereits `UNIQUE`). Wiederholte Aufrufe sind unschädlich.
   - Das Anstoßen läuft über eine dünne Stelle (z. B. `lib/auftraege.js`). In Variante A führt sie die Funktion sofort aus. Bei einem Wechsel zu B legt sie stattdessen einen Auftrag ab, die Aufrufer bleiben unverändert.
   - Der TMDB-Client mit Timeout, Wiederholung und Cache ist ein eigenes Modul. Er ist damit auch von einem Worker oder einem eigenen Dienst nutzbar.

- Kandidat für eine asynchrone Verarbeitung ist das **Nachladen und Aktualisieren von Filmdaten** (`Film.aktualisiert_am`). Dabei ändert sich das API-Design nicht. Suche, Anmeldung und Bewertung bleiben synchron.

4. **Queue: Zielnote 2.0, Queue als Zusatz.** Das Team ist mit einer 2.0 zufrieden (14.10.). Zuerst werden alle Pflichtteile umgesetzt; bleibt danach Zeit, wird Variante B (kleiner Hintergrundauftrag ohne externen Broker) umgesetzt, sonst steht sie als Ausblick im Portfolio. Die Stelle dafür ist unter 3. vorbereitet.

Stand der Umsetzung (07.11.2026)

- **Variante A ist umgesetzt.** TMDB-Client mit Timeout von 5 Sekunden, bis zu drei Versuchen bei Netzwerkfehler, Timeout, 429 und 5xx (Pausen 200 und 400 ms), 404 ohne Wiederholung, sonst 503. Davor liegt ein Cache im Arbeitsspeicher (Suche und Genre-Auswahl 10 Minuten, Filmdetails und Empfehlungen 1 Stunde, Genre-Liste 24 Stunden). Die Empfehlungen zu bis zu zehn Quellfilmen werden mit `Promise.all` gleichzeitig abgefragt.
- **Abweichung von Punkt 3:** Die Funktion heißt nicht `importiereFilm`, sondern `ensureMovie` im `movieService`. Sie ist wie geplant idempotent (Upsert per `ON CONFLICT (tmdb_id) DO UPDATE`) und wird von Detailseite, Bewertungen, Watchlist und Kommentaren gemeinsam genutzt. Eine eigene Auslösestelle (`lib/auftraege.js`) gibt es nicht: `ensureMovie` wird direkt aus den Services aufgerufen. Bei einem Wechsel zu B wäre der Auslöser dort einzubauen.
- **Variante B (Queue) ist nicht gebaut.** Nach Entscheidung 4 hatten zuerst die Pflichtteile Vorrang, danach blieb keine Zeit. Sie steht als Ausblick im Portfolio.
- Seiteneffekt: Weil auch das Lesen von Bewertungen und Kommentaren `ensureMovie` nutzt, kann ein GET einen Film speichern. Abgeschwächt wird das dadurch, dass der Import idempotent ist.

Rückmeldung von Herrn Hartwig (14.10.2026)

Zu den Fragen der Mail vom 13.10. hat Herr Hartwig geantwortet (sinngemäß):

- **REST und GraphQL in einem Server (BFF):** Normalerweise getrennte Server, weil sich Entwicklungswege und Anforderungen schnell trennen. Er würde unsere Lösung aber **akzeptieren**. Im Portfolio die Begründung und das Risiko nennen.
- **Einzelne Fachbereiche als eigene Dienste: Nicht notwendig.** "Wir wollen es nicht übertreiben." Es genügt zu zeigen, dass das Prinzip verstanden und umgesetzt ist.
- **Ausnahme "synchron":** Zwischen Frontend und Backend **eher nicht**, **innerhalb der Backend-Komponenten ok**, zum Nutzer hin **eher nicht**.
- **Queue oder Message-Broker:** Er fragte zurück: "Worüber reden wir hier? Über eine 1.0 oder eine 2.0?" Gemeint sind **Noten**. Sinngemäß: Eine Queue ist keine Pflicht, gehört aber eher zur Bestnote. Das Notenziel des Teams entscheidet.

Unsere Auslegung (unsicher, bei Bedarf nachfragen): Der Weg SPA → BFF darf nicht als synchrone Ausnahme gelten, sondern muss asynchron und nicht blockierend sein. Eine Queue ist nicht verlangt; ob wir sie bauen, hängt vom Notenziel ab. Das Team zielt auf eine 2.0, die Queue kommt nur bei übriger Zeit (siehe Entscheidung 4). Offen ist, ob er mit "asynchron zum Frontend" mehr meint als nicht blockierende Aufrufe (z. B. Server-Sent Events oder WebSocket für Benachrichtigungen).

Betrachtete Alternativen

Alternative	Vorteile	Gründe für oder gegen die Wahl
A: Asynchron im Code, synchroner Weg zwischen den Komponenten mit Begründung	Geringer Aufwand, passt zu Suche, Anmeldung und Bewertung	**Gewählt als erster Schritt.** Risiko: Hartwig versteht "asynchron" strenger.
B: Hintergrundauftrag mit Queue an einer Stelle (z. B. Nachladen von Filmdaten)	Echte Entkopplung, stärkeres Argument im Portfolio, gleichzeitig Beispiel für "Queue" bei der Infrastruktur-Optimierung	**Vorbereitet, noch nicht umgesetzt.** Mittlerer Aufwand. Eine Queue im Speicher geht beim Neustart des Dynos verloren (Aufträge sind idempotent und können neu erzeugt werden). Aufträge in Postgres bräuchten eine neue Tabelle per Migration.
C: Message-Broker zwischen mehreren Diensten (z. B. RabbitMQ)	Vollständig asynchrone Kommunikation	Zu groß für das Projekt: zusätzlicher Dienst, Hosting, Kosten, keine Vorerfahrung.

Folgen

Vorteile

- Der Zeitplan bleibt realistisch, die Nutzerwege bleiben einfach und gut testbar.
- Ein Wechsel zu B betrifft im Wesentlichen eine Stelle (`lib/auftraege.js`) und einen Worker, nicht die Routen und nicht das API-Design.
- Timeout, Wiederholung und Cache sind für den Nachweis einer nicht-funktionalen Anforderung nutzbar (Antwortzeit mit und ohne Cache).

Nachteile und offene Punkte

- Zwischen den Komponenten gibt es zunächst keine echte asynchrone Kommunikation. Ob das für die Bewertung genügt, entscheidet Hartwig.
- Bei B mit Queue im Speicher gehen offene Aufträge beim Neustart eines Dynos verloren. Alternative: Queue in Postgres (neue Migration).
- Die Begründung für den synchronen Weg muss im Portfolio nachvollziehbar stehen.

Betroffene Artefakte: Architekturdiagramm (Kommunikationsarten), Portfolio (Kapitel Implementierung und Reflexion), README von `cinerate-backend`, Mail an Hartwig. `CineRate_API-2.pdf` bleibt unverändert.

Umsetzung

Ursprünglich geplant in `cinerate-backend`: `lib/tmdbClient.js` (Timeout, Wiederholung, Fehlerarten), Cache-Modul um den TMDB-Client, Service-Funktion `importiereFilm`, `lib/auftraege.js` als Auslöser.

Umgesetzt (Stand 07.11.2026): `integrations/tmdbClient.js` (Timeout, Wiederholung, Fehlerarten), `lib/ttlCache.js` und `integrations/cachedTmdbClient.js` (Cache), `ensureMovie` in `services/movieService.js` (idempotenter Import), `Promise.all` in `services/recommendationService.js`. Tests: `test/tmdbClient.test.js` (Wiederholung, Pausen, kein Wiederholen bei 404, mit ausgetauschtem `fetch`), Tests zum Cache in `test/ttlCache.test.js` und `test/cachedTmdbClient.test.js`, zum Filmimport in `test/movieService.test.js`. Die Antwortzeit mit und ohne Cache wurde mit `scripts/messung-cache.js` gemessen (Suche 205 bis 432 ms bei TMDB, unter 0,1 ms aus dem Cache).

ADR-07: MPA-Umsetzung – Express-Server mit Cookie-Login

- **Status:** vom Team bestätigt am 15.10.2026 (Sprintplanung); Stand der Umsetzung nachgetragen am 07.11.2026
- **Datum:** 13.10.2026
- **Beteiligte:** Lukas Krebelder (Entscheidung), Nico Weber (Umsetzung), Madlen Hahn (zu bestätigen)
- **Betrifft:** `cinerate-mpa` (neues Repository), `cinerate-bff`, Portfolio Web-Programmierung (MPA, 5 Punkte)

Kontext

Die Anforderungen von Herrn Hartwig verlangen einen Touchpoint als **MPA** (Multi-Page-Application: jede Seite wird vom Server als eigenes HTML-Dokument geliefert) neben der SPA. Bewertet werden HTML/CSS/JS und Responsivität. Die MPA muss nicht alle Funktionen der SPA bieten; laut Bewertungskriterien soll umgekehrt die SPA die Funktionen der MPA abdecken. Das Repository `cinerate-mpa` ist angelegt (12.10.), Nico baut die MPA, die Heroku-App legt Nico an.

Zu klären waren die Bauweise (statische Seiten oder Server), der Login-Weg und der Zugriff auf die Daten.

Entscheidung

1. **Bauweise B: Express-Server.** Die MPA ist eine kleine Node.js-Anwendung (Express), die pro Seite HTML erzeugt (serverseitiges Rendern mit der Template-Engine EJS). Statische Seiten mit Fetch im Browser (Variante A) wären nur eine zweite SPA und würden den MPA-Gedanken nicht zeigen.
2. **Zugriff nur über das BFF.** Die MPA spricht ausschließlich mit dem BFF, nie direkt mit dem Backend oder der Datenbank. Umgesetzt: Registrieren, Anmelden, Abmelden und Status über REST (`/bff/...`), Filmsuche und Filmdetail über GraphQL (`/bff/graphql`, ADR-01). Das BFF bleibt der einzige Einstiegspunkt für Touchpoints.
3. **Login per Cookie.** Nach der Anmeldung (`POST /bff/auth`) legt der MPA-Server das Token in einem Cookie ab (`HttpOnly`, `Secure`, `SameSite=Lax`). Bei jedem Seitenaufruf liest der MPA-Server das Cookie und sendet das Token als `Authorization: Bearer …` an das BFF. Das Token ist weiterhin das zufällige Token aus ADR-04; Backend und BFF ändern sich nicht. Beim Abmelden ruft der MPA-Server `DELETE /bff/auth` auf und löscht das Cookie.
4. **Umfang:** Registrieren, Anmelden, Abmelden, Filmsuche mit Genrefilter; seit 06.11.2026 zusätzlich die Filmdetailseite. Die MPA soll das MPA-Prinzip zeigen und baut nicht alle Funktionen der SPA nach.
5. **Betrieb:** eigenes Repository `cinerate-mpa`, eigene Heroku-App (Nico). Die BFF-Adresse und die Adresse der MPA stehen als Umgebungsvariablen in der Konfiguration. Das BFF muss die MPA-Adresse nicht für CORS freigeben, weil der Aufruf serverseitig erfolgt.

Begründung

- Entspricht der Anforderung (echte MPA) und zeigt den Unterschied zur SPA sichtbar im Portfolio.
- Das Token landet nicht im JavaScript des Browsers (`HttpOnly`), das ist sicherer als `localStorage`.
- Keine Änderung an Backend und BFF nötig, ADR-04 bleibt gültig.
- Gleiche Technik wie der Rest (Node.js/Express), keine neue Sprache für das Team.

Alternativen

Alternative	Warum nicht
A: statische HTML-Seiten, Daten per Fetch	Verhält sich wie eine SPA, MPA-Prinzip nicht erkennbar
Token im Browser (`localStorage` /Bearer)	Für serverseitig gerenderte Seiten nicht nutzbar, anfälliger für XSS
MPA direkt am Backend	Umgeht das BFF, widerspricht ADR-01

Folgen

- Neue Abhängigkeiten in `cinerate-mpa`: `express`, die Template-Engine `ejs`, `cookie-parser`.
- Cookie `Secure` funktioniert lokal nur über `localhost`; auf Heroku läuft die App über HTTPS.

- Ein Cookie-basierter Login erfordert Schutz vor CSRF bei schreibenden Aktionen; `SameSite=Lax` genügt für die erste Version, Formulare mit POST werden auf denselben Ursprung beschränkt.
- ADR-01 ist um den Touchpoint MPA ergänzt ("zwei Touchpoints, beide über das BFF", nachgetragen am 07.11.2026).
- Aufgaben: Nico baut Grundgerüst und Seiten; Lukas pflegt README und Dokumentation.